



UNIVERSIDAD DE EXTREMADURA

CENTRO UNIVERSITARIO DE MÉRIDA

GRADO EN INGENIERÍA INFORMÁTICA EN
TECNOLOGÍAS DE LA INFORMACIÓN

TRABAJO FIN DE GRADO

DISEÑO, DESARROLLO Y DESPLIEGUE DE
UNA APLICACIÓN WEB FULL-STACK PARA
EL ANÁLISIS DE DATOS DE VIDEOJUEGOS
MEDIANTE EL USO DE API EXTERNA.

FRANCISCO MONTERO SÁNCHEZ

Mérida, septiembre de 2026



UNIVERSIDAD DE EXTREMADURA

CENTRO UNIVERSITARIO DE MÉRIDA

GRADO EN INGENIERÍA INFORMÁTICA EN
TECNOLOGÍAS DE LA INFORMACIÓN

TRABAJO FIN DE GRADO

DISEÑO, DESARROLLO Y DESPLIEGUE DE
UNA APLICACIÓN WEB FULL-STACK PARA
EL ANÁLISIS DE DATOS DE VIDEOJUEGOS
MEDIANTE EL USO DE API EXTERNA.

Autor: Francisco Montero Sánchez

Fdo:

Director: José Manuel Chaves González

Fdo:

Resumen

League of Legends es un videojuego competitivo en el que millones de jugadores compiten diariamente para mejorar su rendimiento y alcanzar rangos superiores. La gran cantidad de campeones, objetos, roles y estrategias, junto con los cambios introducidos en cada parche, hace necesario disponer de información actualizada y fácil de consultar. Los jugadores necesitan acceder rápidamente a estos datos para conocer su rendimiento y tomar mejores decisiones durante las partidas.

Este Trabajo de Fin de Grado presenta EasyRift, una aplicación web full-stack que obtiene datos mediante la API de Riot Games, los almacena, los organiza y los analiza para convertirlos en información útil para el usuario. La aplicación permite buscar jugadores de diferentes plataformas y consultar sus perfiles, rangos clasificatorios, estadísticas, partidas recientes y partidas en curso. También muestra estadísticas globales y una clasificación de campeones por tiers, además de builds recomendadas para cada combinación de campeón y rol. Por último, añade un apartado dedicado a los jugadores profesionales, con sus equipos, perfiles y builds clasificatorias recientes.

El desarrollo se lleva a cabo mediante tres incrementos. El primero establece un producto mínimo viable con las funciones básicas de consulta. El segundo añade estadísticas, rangos y procesos automáticos. El tercero agrega la clasificación de campeones, las recomendaciones por rol y los perfiles profesionales. La aplicación utiliza Java y Spring Boot para el backend, React y TypeScript para el frontend, y PostgreSQL para almacenar la información. Los procesos programados actualizan periódicamente a los profesionales y recalculan los datos derivados de las partidas almacenadas.

La versión final se despliega mediante Vercel y Render y es accesible a través de un dominio propio. Las pruebas funcionales realizadas permiten comprobar que las funciones implementadas se comportan de la forma esperada y cumplen los requisitos definidos. Como resultado, se obtiene una aplicación funcional que presenta la información de forma sencilla y proporciona una base para futuras ampliaciones.

Nota: Muchos de los conceptos que salen aquí son explicados en la memoria la primera vez que salen. ¿Debería asumir que esto es la primera vez que salen y explicarlos o lo dejo como está y ya se explica en la memoria? Por ejemplo la palabra campeón y build no se entienden si no tienes el contexto. En el caso del MVP lo pongo entero sin siglas y ya en la memoria la primera vez que sale se explica

Palabras clave - aplicación web, League of Legends, API REST, Riot Games, Spring Boot, React, PostgreSQL, análisis de datos.

Abstract

League of Legends is a competitive video game in which millions of players compete every day to improve their performance and reach higher ranks. The large number of champions, items, roles and strategies, along with the changes introduced in each patch, means that players need quick and easy access to updated information. This information helps them to understand their performance and make better decisions during matches.

This project presents EasyRift, a full-stack web application that gets data from the Riot Games API and stores, organises and analyses them to provide useful information to users. The application allows users to search for players from different platforms and access their profiles, ranks, statistics, recent matches and current matches. It also displays global champion statistics, a tier-based champion ranking and recommended builds for each champion and role combination. Finally, it includes a section dedicated to professional players, containing their teams, profiles and recent ranked builds.

The development process is divided into three increments. The first establishes a minimum viable product with the basic query features. The second adds statistics, ranks and automated processes. The third incorporates champion rankings, role-based recommendations and professional player profiles. The application uses Java and Spring Boot for the backend, React and TypeScript for the frontend, and PostgreSQL for data storage. Scheduled processes periodically update professional players and recalculate the information derived from stored matches.

The final version is deployed using Vercel and Render and is accessible through its own domain. The functional tests confirm that the implemented features behave as expected and meet the defined requirements. The result is a functional application that presents information clearly and provides a foundation for future extensions.

Keywords - web application, League of Legends, REST API, Riot Games, Spring Boot, React, PostgreSQL, data analysis

Índice general

Índice de figuras	VIII
Índice de tablas	X
1. Introducción	1
1.1. Introducción	1
1.2. Motivación	3
1.3. Objetivos	4
1.4. Metodología	6
1.5. Estructura del documento	7
2. Estado del arte	8
2.1. Aplicaciones y herramientas existentes	8
2.2. Plataformas y servicios utilizados	9
2.3. Tecnologías empleadas en este tipo de aplicaciones	10
2.3.1. Tecnologías frontend	11
2.3.2. Tecnologías backend	13
2.3.3. Persistencia y acceso a datos	14
2.3.4. Despliegue	15
2.4. Limitaciones de las soluciones existentes	16
2.5. Aportación de este trabajo	17
3. Marco teórico	19
3.1. Conceptos básicos de League of Legends	19

3.2.	Obtención y tratamiento de datos	21
3.2.1.	API de Riot	21
3.2.2.	Riot ID	22
3.2.3.	Plataformas y regiones	23
3.2.4.	Partidas y participaciones	24
4.	Análisis del problema	25
4.1.	Planteamiento del problema	25
4.2.	Requisitos del sistema	26
4.3.	Arquitectura general y componentes del sistema	30
4.4.	Flujo de información	31
5.	Diseño del sistema	35
5.1.	Diseño funcional e incremental	35
5.1.1.	Primer incremento: MVP	36
5.1.2.	Segundo incremento: estadísticas y automatización . . .	37
5.1.3.	Tercer incremento: clasificación y recomendaciones . . .	38
5.2.	Tecnologías seleccionadas	39
5.2.1.	Base de datos	40
5.2.2.	Backend	41
5.2.3.	Frontend	41
5.2.4.	Servicios externos	42
5.2.5.	Despliegue	42
5.3.	Diseño de la base de datos	43
5.4.	Diseño del backend	45
5.4.1.	Diseño por capas	45
5.4.2.	Diseño de la API REST	46
5.4.3.	Sincronización de jugadores	47
5.4.4.	Procesos automáticos	49
5.4.5.	Cálculo de estadísticas y recomendaciones	49
5.5.	Diseño del frontend	50

5.5.1.	Estructura y navegación	50
5.5.2.	Diseño basado en componentes	52
5.5.3.	Adaptabilidad y accesibilidad	52
6.	Desarrollo incremental	54
6.1.	Preparación y organización del proyecto	54
6.2.	Implementación de la persistencia de datos	56
6.2.1.	Entidades y relaciones	56
6.2.2.	Repositorios y consultas	58
6.3.	Desarrollo del primer incremento	59
6.3.1.	Comunicación con Riot Games	59
6.3.2.	Consulta y sincronización de jugadores	60
6.3.3.	API y transferencia de datos	61
6.3.4.	Primera interfaz web	62
6.4.	Desarrollo del segundo incremento	63
6.4.1.	Rangos y estadísticas de jugadores	64
6.4.2.	Estadísticas de campeones	64
6.4.3.	Automatización	65
6.4.4.	Ampliación del frontend	66
6.5.	Desarrollo del tercer incremento	66
6.5.1.	Inicialización de profesionales	66
6.5.2.	Clasificación de campeones	67
6.5.3.	Generación de builds recomendadas	68
6.5.4.	Interfaz final	69
6.6.	Despliegue a producción	69
7.	Evaluación de la aplicación final	71
7.1.	Entorno y método de evaluación	71
7.2.	Validación funcional mediante casos de uso	72
7.2.1.	Consulta de perfiles de jugadores	72
7.2.2.	Consulta de campeones	76

7.2.3. Consulta de jugadores profesionales	79
7.2.4. Procesos automáticos y base de datos	81
7.3. Cumplimiento de requisitos	83
7.4. Limitaciones	86
8. Conclusiones y trabajo futuro	90
8.1. Conclusiones y conocimientos adquiridos	90
8.2. Aportaciones e impacto de EasyRift	92
8.3. Líneas de trabajo futuro	95
8.3.1. Cuarto incremento: ampliación de datos y funcionalidades	95
8.3.2. Quinto incremento: preparación para producción	96
8.3.3. Evolución a largo plazo	97
Referencias	99
A. Uso responsable de herramientas de inteligencia artificial	102

Índice de figuras

2.1. Encuesta sobre frameworks y tecnologías web. Fuente: Stack Overflow [1]	12
2.2. Encuesta sobre bases de datos. Fuente: Stack Overflow [1] . . .	15
3.1. DTO devuelto por la API de Riot. Fuente: Riot Games [2] . .	22
3.2. Endpoint <code>MATCH-V5</code> . Fuente: Riot Games [2]	23
4.1. Diagrama de componentes de alto nivel	30
4.2. Diagrama de flujo. Perfil de usuario	32
4.3. Diagrama de flujo. Builds de profesionales	33
5.1. Diagrama de casos de uso	36
5.2. Diagrama entidad-relación final	44
5.3. Estructura de navegación del frontend	51
6.1. Organización simplificada del proyecto	55
7.1. Buscador de jugadores	72
7.2. Perfil de jugador	73
7.3. Consulta de un jugador de la plataforma coreana	74
7.4. Consulta de una partida en curso	75
7.5. Detalles de partida	76
7.6. Clasificación de campeones por tiers	77
7.7. Detalle de un campeón y selección de rol	78
7.8. Detalle de un campeón en otro rol	78
7.9. Información detallada de un objeto	79

7.10. Jugadores profesionales agrupados por equipos	80
7.11. Perfil de un jugador profesional	81
7.12. Ejecución de los procesos automáticos en Render	82
7.13. Jugadores almacenados en PostgreSQL	82
7.14. Adaptación de la interfaz a una pantalla móvil	84
7.15. Búsqueda incorrecta	85
8.1. Alternativas de objetos mostradas por OP.GG. Fuente: OP.GG [3]	92
8.2. Recomendación personalizada de campeones en fase beta de OP.GG. Fuente: OP.GG [4]	93
8.3. Información de un jugador profesional mostrada por LolPros. Fuente: LolPros [5]	94

Índice de tablas

4.1. Requisitos funcionales.	27
4.2. Requisitos no funcionales.	28
4.3. Requisitos funcionales de ampliación.	29
5.1. Tecnologías utilizadas en EasyRift	40
5.2. Endpoints de la API REST	47
7.1. Resumen del cumplimiento de requisitos	86

Capítulo 1

Introducción

1.1. Introducción

La industria de los videojuegos ha crecido enormemente en los últimos años. Desde la aparición de los primeros títulos en los años 50, estos han ido evolucionando constantemente, mejorando aspectos gráficos, de jugabilidad e incluso descubriendo nuevos géneros.

A comienzos del siglo, el aumento del público interesado en los videojuegos llevó a que se crearan ligas y torneos similares a los existentes en los deportes tradicionales. Así surgieron los deportes electrónicos, que se consolidaron como una disciplina competitiva que exige la actualización constante de los jugadores.

Este proyecto se centra en *League of Legends* (LoL), uno de los videojuegos más jugados en la actualidad. Se trata de un videojuego competitivo perteneciente al género *multiplayer online battle arena* (MOBA), donde dos equipos formados por cinco jugadores, distribuidos en diferentes roles, compiten por destruir la base del rival.

Nota: Me he vuelto un poco loco con el formato, ya que no tengo una referencia clara y cada TFG lo hace de una forma diferente. Al final, el criterio que he seguido es el siguiente (Si algo está mal dímelo y lo cambio):

- La primera vez que aparece una sigla que no suena bien en español, se escribe el término original en cursiva en inglés seguido de la sigla entre paréntesis. En las apariciones posteriores se utiliza únicamente la sigla. Traducir MOBA al español

como que no tiene mucho sentido porque se usa MOBA y ya, pero entiendo que tengo que explicar las siglas.

- Cuando el término se utiliza en español, se pone directamente en español, como producto mínimo viable (MVP) o sistema gestor de bases de datos (SGBD).
- Si las siglas son típicas como PDF, JSON, URL o TFG no se pone lo que significan porque se da por conocimiento general.
- Los títulos de videojuegos y los nombres de los casos de uso se escriben en cursiva.
- Las clases, métodos, anotaciones, archivos y rutas se ponen en monoespaciado.
- Los términos técnicos utilizados habitualmente, como frontend, backend, full-stack, build, tier, endpoint y framework, se dejan normal para evitar una cantidad excesiva de cursivas. Los nombres oficiales de empresas y tecnologías también. Ej: Riot Games o Data Dragon.
- Siempre que existe una alternativa que suena bien en español, se utiliza en vez del término inglés.
- Las referencias Figura, Tabla y Algoritmo se dejan con la primera en mayúsculas, mientras que capítulo, sección y ecuación se dejan en minúscula.

Los MOBA son videojuegos con una multitud de combinaciones posibles. Entre ellos, LoL consta de un amplio conjunto de campeones (personajes jugables), habilidades y objetos, lo que genera un amplio espacio de decisiones para el jugador.

El elevado número de combinaciones posibles, junto con la actualización constante de objetos y personajes, hace que la complejidad del videojuego sea muy elevada para los jugadores casuales que no pueden dedicar el tiempo necesario para mantenerse actualizados de forma constante.

Para afrontar este problema, la empresa dueña del videojuego, Riot Games, proporciona una *Application Programming Interface* (API) que permite obtener datos relacionados con cuentas, partidas y otros elementos del juego. Esto permite la creación de aplicaciones que analizan las mejores combinaciones posibles en cada momento, facilitando la experiencia del usuario casual.

Este trabajo se basa en la creación de una aplicación web que permite consultar perfiles de jugadores, visualizar sus rangos, estadísticas e historial de partidas, comprobar si se encuentran jugando y analizar la información de los campeones. Además, la aplicación clasifica los campeones mediante tiers, genera builds (conjuntos de objetos comprados) recomendadas por rol y permite consultar los perfiles y las partidas recientes de jugadores profesionales.

El desarrollo de esta aplicación permitirá a los usuarios consultar las mejores opciones posibles en tan solo unos segundos. De esta forma, podrán consultar información relevante de forma rápida, reduciendo el esfuerzo necesario para mantenerse actualizados y facilitando la toma de decisiones dentro del videojuego.

Dado el alcance del problema y las casi infinitas funcionalidades que se pueden implementar, el desarrollo del proyecto se ha planteado mediante una metodología iterativa e incremental. De esta forma, en una primera iteración se desarrolla el producto mínimo viable (MVP) y en las siguientes se implementan funcionalidades de interés para el usuario final.

1.2. Motivación

Tras conocer y analizar el entorno de los videojuegos competitivos, específicamente los MOBA, durante más de una década, se ha observado la necesidad de que la comunidad cuente con un sitio fijo al que acudir en busca del conocimiento necesario.

A diferencia de otros videojuegos, como los *first-person shooter* (FPS), como *Counter-Strike* o los *Battle Royale*, como *Fortnite*, los MOBA dan mucha importancia a la estrategia, la coordinación del equipo y la toma de decisiones durante la partida. Por este motivo, suelen presentar una barrera de entrada más elevada y requieren un mayor tiempo de aprendizaje.

La motivación principal de este proyecto es facilitar el aprendizaje continuo de los jugadores experimentados del LoL y reducir la barrera de entrada para los jugadores nuevos que, sin ayuda, podrían tardar años en aprender los fundamentos del juego y estar al día.

La complejidad al aprender los diversos aspectos del videojuego hace que existan otros factores que motivan la existencia de este proyecto:

- **Conocimiento del metajuego:** Con la actualización constante de los videojuegos competitivos, surge el meta. En un contexto donde cada dos semanas cambian los personajes, objetos y características del entorno,

surge la necesidad de conocer las mejores opciones en todo momento. La forma más rápida de satisfacer esto es tener un sitio al que acudir cuando se desee conocer el meta actual.

- **Revisión de estadísticas:** La mejora en un entorno competitivo requiere un análisis detallado del jugador. Muchas veces, es necesario consultar estadísticas avanzadas, como tendencias, puntos fuertes y débiles, o el desempeño en cada partida.
- **Centralización de la información:** La información actual recopilada por terceros se encuentra muy distribuida en diferentes aplicaciones. Unas se especializan en consultar perfiles, otras analizan el rendimiento del jugador mediante inteligencia artificial y otras ofrecen recomendaciones a partir de los datos existentes, pero no existe un sitio al que los jugadores, especialmente los más novatos, puedan acceder para consultar la información básica de una forma sencilla.

La existencia de un sitio al que acudir en busca del conocimiento de forma sencilla satisface todos estos factores y es de gran ayuda para la comunidad.

Junto con las motivaciones generales de la existencia de este proyecto, también surge una motivación personal en la forma en que se llevará a cabo.

A lo largo de la carrera se han estudiado distintas tecnologías y fundamentos del desarrollo software. Este trabajo permite complementar esa base con herramientas actuales utilizadas habitualmente en el desarrollo de aplicaciones web reales. Actualmente, la mayor parte de las aplicaciones y servicios web se basan en el uso de API para el intercambio de información. Además, el proyecto ofrece la oportunidad de desarrollar una aplicación full-stack que integra frontend, backend y base de datos [6].

1.3. Objetivos

El objetivo principal es diseñar, desarrollar y desplegar una aplicación web full-stack orientada a la consulta y análisis de información del videojuego *League of Legends*, a partir de datos obtenidos de la API pública de Riot Games, siguiendo una metodología de desarrollo iterativa e incremental.

Para lograr el objetivo principal, se han definido los siguientes objetivos específicos:

- **Diseñar un modelo de datos** para guardar la información de los jugadores, partidas, campeones, builds y jugadores profesionales. Este

modelo de datos sirve para que la información pueda reutilizarse sin repetir las mismas consultas a la API externa.

- **Desarrollar un backend** que se conecte con la API de Riot, procese la información recibida, la almacene en la base de datos y la exponga mediante una API basada en *representational state transfer* (REST).
- **Desarrollar un frontend web** que permita al usuario final consultar perfiles, historial de partidas y builds de jugadores profesionales de una forma clara y sencilla.
- **Implementar un primer incremento funcional** que actúe como MVP. De esta forma, se dispone de una base sobre la que agregar funcionalidades en los siguientes incrementos.
- **Configurar un entorno de ejecución local** que permita levantar los distintos componentes de la aplicación, incluyendo frontend, backend y base de datos.
- **Completar el desarrollo de una aplicación web**, desde su análisis y diseño inicial hasta su implementación, integración y despliegue público, reuniendo en un único proyecto los conocimientos adquiridos durante el grado.

Además, para complementar la base del proyecto, se ha establecido una serie de objetivos de ampliación:

- **Añadir rangos clasificatorios y estadísticas de campeones y jugadores** a partir de las partidas y participaciones almacenadas.
- **Automatizar la actualización de la información** mediante tareas programadas que actualicen periódicamente los profesionales y los datos calculados de los campeones.
- **Clasificar los campeones mediante tiers** a partir de las partidas clasificatorias del parche actual.
- **Recomendar builds por campeón y rol** a partir de las partidas clasificatorias del parche actual.
- **Habilitar la consulta de jugadores profesionales** mediante un listado organizado por equipos y perfiles profesionales.
- **Desplegar la aplicación en producción** publicando el frontend, el backend y la base de datos en servicios de hosting y configurando un dominio propio para su acceso público.

1.4. Metodología

El desarrollo del proyecto se ha llevado a cabo siguiendo una metodología de desarrollo iterativa e incremental. Esta metodología permite construir una base funcional desde el primer incremento y ampliar posteriormente las funciones de la aplicación sin rehacer completamente la arquitectura base del sistema.

La elección de esta metodología se debe a que el desarrollo del proyecto conlleva el aprendizaje de nuevas herramientas y tecnologías. En caso de haber usado una metodología tradicional como el desarrollo en cascada, se podrían haber planificado o diseñado elementos que posteriormente resultasen difíciles de cumplir. En cambio, al adoptar un enfoque iterativo e incremental se consigue solucionar este problema avanzando de forma progresiva con incrementos e iteraciones.

Nota: No sé si poner aquí algo como: no se han elegido metodologías novedosas ágiles como Scrum porque están diseñadas para trabajar en equipo y yo estoy solo.

El desarrollo evoluciona mediante distintos incrementos funcionales. Cada incremento añade nuevas funcionalidades a la versión previa del sistema; de esta manera, el proyecto crece de forma progresiva en lugar de desarrollarse completamente desde el principio. Esto permite establecer una base funcional que se puede mejorar y completar con el transcurso de los incrementos.

A su vez, cada incremento se desarrolla de forma iterativa. En cada uno de ellos se siguen todas las etapas del proceso de creación de software: análisis, diseño, implementación y validación. Gracias a este proceso no solo se añaden nuevas funcionalidades, sino que también se puede refinar la base anterior en caso de ser necesario [7].

El primer incremento del proyecto se centra en el desarrollo del MVP. El MVP consiste en una versión inicial del software que incluye solo las características esenciales para satisfacer al usuario y evaluar la viabilidad del proyecto. De esta forma, el primer incremento construye una primera versión funcional que sirve como base para su posterior evolución [8].

En este primer incremento se establece la arquitectura general de la aplicación: se crea el modelo de datos, la integración con la API de Riot y la estructura básica del backend y del frontend. A partir de esta base se podrán incluir nuevas funcionalidades de una forma más rápida y sencilla.

El segundo incremento añade los rangos y las estadísticas de jugadores

y campeones, además de automatizar la actualización de la información mediante tareas programadas. El tercero agrega la clasificación de campeones por tiers, las builds recomendadas por rol y la consulta de profesionales junto con sus perfiles.

1.5. Estructura del documento

Este documento se estructura en los siguientes capítulos:

- **Capítulo 2. Estado del arte:** Revisa las aplicaciones existentes basadas en API de videojuegos y las tecnologías más utilizadas en este tipo de sistemas. Además, se expone la aportación de este trabajo respecto a las soluciones ya existentes.
- **Capítulo 3. Marco teórico:** Introduce los conceptos generales y el vocabulario técnico relacionados con el proyecto con el objetivo de aportar una base de conocimiento para entender el trabajo realizado.
- **Capítulo 4. Análisis del problema:** Describe el problema que se pretende resolver, los requisitos de la aplicación, los componentes del sistema y la arquitectura general.
- **Capítulo 5. Diseño del sistema:** Detalla las tecnologías utilizadas y justifica las decisiones tomadas durante el diseño. También se describen la base de datos, el backend, la API REST, el frontend y el despliegue de la solución.
- **Capítulo 6. Desarrollo incremental:** Explica la evolución del proyecto a través de los sucesivos incrementos, describiendo las funcionalidades implementadas en cada uno de ellos y la mejora respecto a la versión anterior.
- **Capítulo 7. Evaluación de la aplicación final:** Presenta el despliegue de la versión final, sus características técnicas y distintos casos de uso para validar el funcionamiento de la aplicación.
- **Capítulo 8. Conclusiones y trabajo futuro:** Recoge las principales conclusiones del trabajo, las aportaciones realizadas y los conocimientos adquiridos durante su desarrollo. Finalmente, se plantean posibles mejoras para la evolución del proyecto.

Capítulo 2

Estado del arte

En este capítulo se revisan las aplicaciones y herramientas existentes relacionadas con el análisis de datos de videojuegos, especialmente en el caso del LoL. También se presentan las plataformas y servicios más utilizados, así como las tecnologías más habituales en el desarrollo de este tipo de sistemas. El objetivo final es situar este trabajo en el contexto actual y justificar su aportación frente a las soluciones existentes.

2.1. Aplicaciones y herramientas existentes

En el contexto actual del LoL, existen numerosas aplicaciones orientadas a satisfacer las distintas necesidades de los jugadores. Algunas de ellas están centradas en consultas de perfiles e historiales, permitiendo buscar jugadores y revisar su historial reciente. Entre este tipo de aplicaciones, la más destacada es OP.GG, que se ha consolidado como la mayor referencia para el seguimiento de cuentas y la consulta rápida de información. Esta plataforma comenzó a funcionar en el año 2012 como buscador de estadísticas del LoL y actualmente cuenta con 16 videojuegos diferentes, sumando 14 millones de visitas diarias y más de 55 millones de visitantes únicos mensuales [9].

Junto a estas herramientas, también existen otras aplicaciones especializadas en campeones, builds y meta. Su función principal es mostrar las mejores combinaciones de objetos, runas, orden de habilidades y estadísticas para cada campeón. Este tipo de aplicaciones es realmente útil para estar actualizado en todo momento y elegir la mejor combinación. Una de las más destacadas es Blitz.gg, cuyo perfil de LinkedIn indica que la aplicación cuenta con más de 8 millones de usuarios activos [10]. Además, este tipo de apli-

caciones suele ofrecer un overlay que ayuda a tomar decisiones en tiempo real.

Nota: No sé si poner como fuente el linkedin de la empresa es fiable, no he encontrado una fuente "fiable" de información.

Por último, también existen herramientas orientadas al seguimiento de jugadores profesionales. Estas permiten conocer los equipos, roles y cuentas utilizadas por los profesionales. Entre estas plataformas destaca LolPros, que reúne información sobre jugadores profesionales y las diferentes cuentas asociadas a cada uno [11].

Las cifras de uso de plataformas como OP.GG y Blitz.gg muestran la importancia que tienen estas herramientas dentro de la comunidad. La información necesaria para mejorar y mantenerse actualizado no se encuentra dentro del propio videojuego, por lo que muchos jugadores recurren a aplicaciones externas para consultar su rendimiento, conocer el estado actual del juego y decidir qué campeones, objetos o estrategias utilizar.

2.2. Plataformas y servicios utilizados

El funcionamiento de este tipo de aplicaciones depende en gran medida de la disponibilidad de fuentes de datos fiables y actualizadas. En el caso de LoL, la fuente principal de información es la API de Riot, un servicio oficial que permite acceder a información relacionada con cuentas de usuarios, partidas y diferentes elementos del juego. Gracias a esta API, los desarrolladores pueden construir aplicaciones y herramientas externas capaces de procesar la información del juego para satisfacer las necesidades de la comunidad del videojuego.

La API de Riot utiliza un sistema de *API Keys* para autenticar las solicitudes realizadas por aplicaciones externas. Riot ofrece diferentes tipos de claves según el uso y la fase de desarrollo del producto:

- ***Development API Key***: es la clave más básica y se puede obtener gratuitamente desde la web oficial de Riot Developer utilizando una cuenta de Riot. Esta clave está pensada para pruebas y desarrollo inicial, por lo que tiene una limitación de 100 llamadas a la API cada 2 minutos y, además, debe renovarse diariamente.
- ***Personal API Key***: está orientada a proyectos personales, pequeñas comunidades privadas o trabajos de investigación. Para obtenerla, es

necesario registrar el producto en Riot Developer y proporcionar una descripción detallada del proyecto. A diferencia de la clave de desarrollo, no debe renovarse diariamente, pero sigue teniendo los mismos límites y no puede utilizarse para aplicaciones públicas.

- ***Production API Key***: está destinada a aplicaciones públicas en producción. Para obtener acceso a esta clave, es necesario enviar una solicitud a Riot Games explicando el funcionamiento de la aplicación, una descripción detallada del proyecto y la URL de la aplicación en producción. Este tipo de clave tiene límites mucho más amplios, pensados para soportar el tráfico de usuarios reales. En caso de llevar el proyecto a producción, sería necesario solicitar esta clave.

Las características y restricciones de estas claves se encuentran definidas en la documentación para desarrolladores de Riot Games [12].

Además de la API de Riot, también es necesario utilizar Data Dragon como fuente de recursos estáticos. Este servicio proporciona información y recursos visuales del juego, como campeones, objetos o iconos de perfil. A diferencia de la API de Riot, que permite obtener datos, Data Dragon se utiliza para acceder a la información estática del videojuego.

Estos recursos están organizados por versión del juego e idioma, lo que permite trabajar con datos actualizados a cada parche en el lenguaje deseado. Por este motivo, Data Dragon es muy útil en aplicaciones de análisis o consulta de datos del LoL, ya que permite complementar la información obtenida mediante la API con nombres, descripciones e imágenes que mejoran la presentación de los datos [12].

2.3. Tecnologías empleadas en este tipo de aplicaciones

El desarrollo de aplicaciones web ha evolucionado mucho desde las primeras páginas estáticas hasta los sistemas actuales, donde es habitual separar la interfaz de usuario, la lógica del servidor y el acceso a datos. En este contexto, las API REST se han convertido en una de las formas más utilizadas para comunicar aplicaciones a través de internet, ya que están basadas en interacciones cliente-servidor mediante peticiones HTTP [13].

Este modelo encaja perfectamente en aplicaciones basadas en videojuegos, ya que muchas de ellas dependen de API externas para acceder a datos

del propio juego. Un ejemplo claro es la API de Riot Games, mencionada anteriormente, que permite obtener información actualizada sobre cuentas, partidas y otros elementos de LoL.

En este tipo de aplicaciones, suele emplearse una arquitectura full-stack, separando la interfaz de usuario, la lógica del servidor y el almacenamiento de datos. Esta separación facilita el mantenimiento del sistema y permite que cada parte se desarrolle de forma independiente.

Nota: Todo esto de full stack ya lo expliqué tal cual al final de la motivación, no sé si repetir algo está bien o digo algo como "como se mencionó anteriormente" igual que hice con la descripción de la API de Riot en el parrafo anterior.

2.3.1. Tecnologías frontend

En la parte frontend, la tendencia más común es el desarrollo de aplicaciones de una sola página (SPA). En este modelo, el navegador carga un único documento principal y el contenido se actualiza dinámicamente mediante JavaScript, sin necesidad de recargar la página en cada interacción [14].

Para desarrollar este tipo de interfaces se utilizan tecnologías como React, Vue o Angular. Estas herramientas permiten organizar la interfaz mediante componentes reutilizables, lo que facilita la división de la aplicación en partes más pequeñas. Este enfoque resulta útil en aplicaciones que obtienen datos de API externas, ya que permite actualizar de forma controlada las partes de la interfaz que dependen de la información consultada.

Nota: No sé hasta qué punto debo poner fuentes. Pongo a continuación de donde saco que son las más utilizadas, aunque es sabido por todo el mundo sin necesidad de tener una fuente, la cosa es, debería poner también la web de cada tecnología como fuente de que sirven para interfaces basadas en componentes? lo veo como poner por poner.

A partir de la encuesta general de tecnologías web realizada por Stack Overflow en 2025, se puede ver el uso de algunas de las principales tecnologías utilizadas en frontend. Como muestra la Figura 2.1, React aparece con un 44,7 % de uso entre los más de 23 000 encuestados, mientras que Angular alcanza un 18,2 % y Vue.js un 17,6 %. Aunque la encuesta no se limita únicamente al desarrollo frontend e incluye otras tecnologías de backend, estos

datos establecen a React, Angular y Vue.js como las opciones más relevantes dentro del desarrollo de interfaces web modernas.

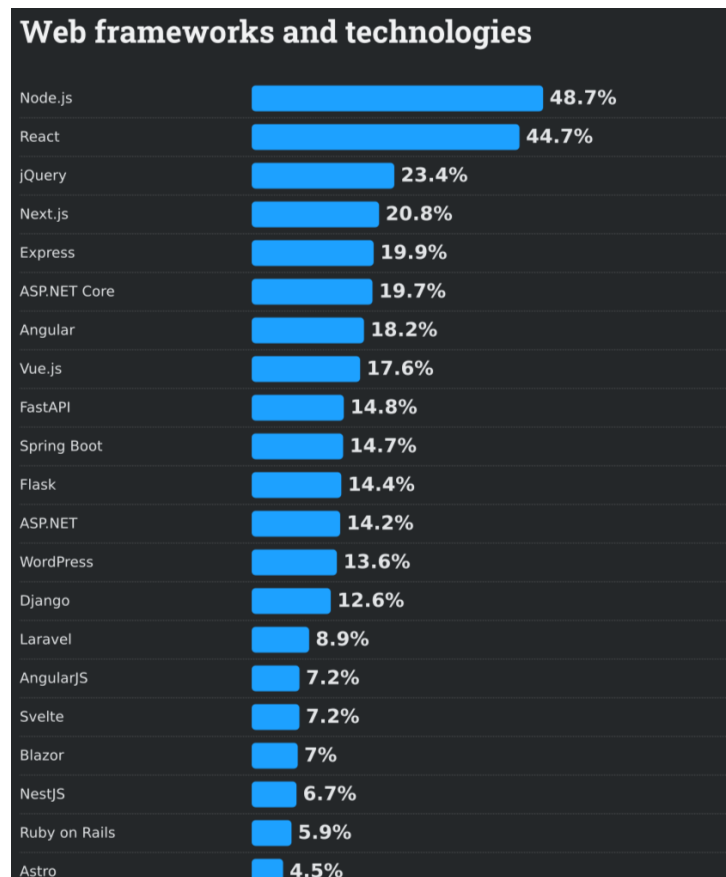


Figura 2.1: Encuesta sobre frameworks y tecnologías web. Fuente: Stack Overflow [1]

Nota: Al poner la fuente en el pie de figura y al tener la memoria un índice de figuras, sale como [1] aunque no es la primera. ¿Eso está bien?

Junto a JavaScript, también es frecuente el uso de TypeScript. Esta tecnología añade sintaxis de tipos sobre JavaScript, lo que ayuda a detectar errores durante el desarrollo y mejora la mantenibilidad del código. En aplicaciones que usan API externas, el uso de tipos permite representar de forma más clara la estructura de los datos recibidos.

Nota: Aquí lo mismo, pongo fuente de que es javascript y que es typescript? es que si me lio a poner referencias acabo con 10 páginas de referencias.

Además del framework o librería principal, las aplicaciones frontend suelen utilizar herramientas de construcción como Vite. Estas herramientas permiten ejecutar el proyecto durante el desarrollo de forma rápida y generar una versión optimizada para producción. Para ello, Vite incluye un servidor de desarrollo con recarga rápida de módulos y un sistema de compilación que empaqueta el código. Además, puede integrarse con distintos frameworks y ampliarse mediante plugins [15].

En cuanto al diseño visual, existen distintas formas para construir una interfaz de calidad. Frameworks como Bootstrap ofrecen componentes y estilos preparados para crear interfaces responsive, mientras que herramientas como Tailwind CSS proporcionan clases que permiten construir diseños personalizados directamente desde el código HTML. Estas herramientas son muy útiles para tener una interfaz de calidad sin necesidad de escribir grandes hojas de estilo CSS.

2.3.2. Tecnologías backend

En la parte backend, lo más común es construir una API propia que actúe como intermediaria entre el frontend, la base de datos y las API externas. De esta forma, el frontend no se comunica directamente con los servicios externos, sino que lo hace a través del backend de la aplicación. Esta división permite centralizar la lógica de negocio en un mismo lugar para controlar mejor los errores, gestionar la persistencia de datos y proteger información sensible.

Para desarrollar backends, existen varias tecnologías ampliamente utilizadas. Dentro del ecosistema de Java, destaca Spring Boot, un framework para crear aplicaciones y servicios web. También son frecuentes aplicaciones basadas en Node.js, como Express; frameworks de Python, como FastAPI; y tecnologías del entorno .NET, como ASP.NET Core.

Como se muestra en la Figura 2.1, la encuesta de Stack Overflow de 2025 también refleja el uso de varias de estas tecnologías: Node.js aparece con un 48,7 %, Express con un 19,9 %, ASP.NET Core con un 19,7 %, FastAPI con un 14,8 %, y Spring Boot con un 14,7 %. Estos datos muestran las diferentes alternativas para el desarrollo de aplicaciones web y API en ecosistemas como JavaScript, Java, Python o .NET.

Además de elegir una tecnología concreta, en el desarrollo backend también es importante establecer una organización adecuada. En aplicaciones pequeñas puede encontrarse toda la lógica en una única capa, mezclando la gestión de peticiones, el procesamiento de datos y el acceso a la base de da-

tos. Aunque esto puede ser válido para prototipos, dificulta el mantenimiento cuando la aplicación crece.

Por este motivo, en los backends modernos es habitual separar responsabilidades mediante una arquitectura por capas. Normalmente se distinguen controladores, encargados de gestionar las peticiones HTTP; servicios, donde se concentra la lógica de negocio; repositorios, encargados del acceso a datos; y modelos o entidades, que representan la información del sistema. En aplicaciones basadas en API externas, también pueden añadirse componentes específicos para comunicarse con dichos servicios y el patrón *Data Transfer Object* (DTO) para controlar la estructura de los datos que se reciben o envían.

2.3.3. Persistencia y acceso a datos

Las bases de datos permiten almacenar información para ser reutilizada posteriormente. En el desarrollo de aplicaciones web es habitual encontrar bases de datos relacionales como PostgreSQL, MySQL o MariaDB, que organizan la información en tablas y permiten definir relaciones entre ellas. Este modelo resulta muy útil cuando los datos tienen una estructura clara y están relacionados entre sí, como puede ocurrir con usuarios, estadísticas, partidas o elementos asociados a una cuenta.

Además de las bases de datos relacionales, existen bases de datos NoSQL como MongoDB, que pueden ser útiles cuando la información tiene una estructura flexible que cambia frecuentemente. También son útiles las bases de datos en memoria, como Redis, para almacenar temporalmente información consultada con frecuencia. Esto permite reducir consultas repetidas a la base de datos, algo clave en las aplicaciones que dependen de API con límites de uso.

Como se puede ver en la Figura 2.2, la encuesta de Stack Overflow de 2025 muestra que PostgreSQL es una de las bases de datos más utilizadas, con un 55,6 % de uso entre los encuestados. También aparecen MySQL, con un 40,5 %, SQLite, con un 37,5 %, Microsoft SQL Server, con un 30,1 %, Redis con un 28 % y MongoDB, con un 24 %. Estos datos muestran que las bases de datos relacionales son las más utilizadas, aunque también se utilizan bases de datos NoSQL o en memoria.

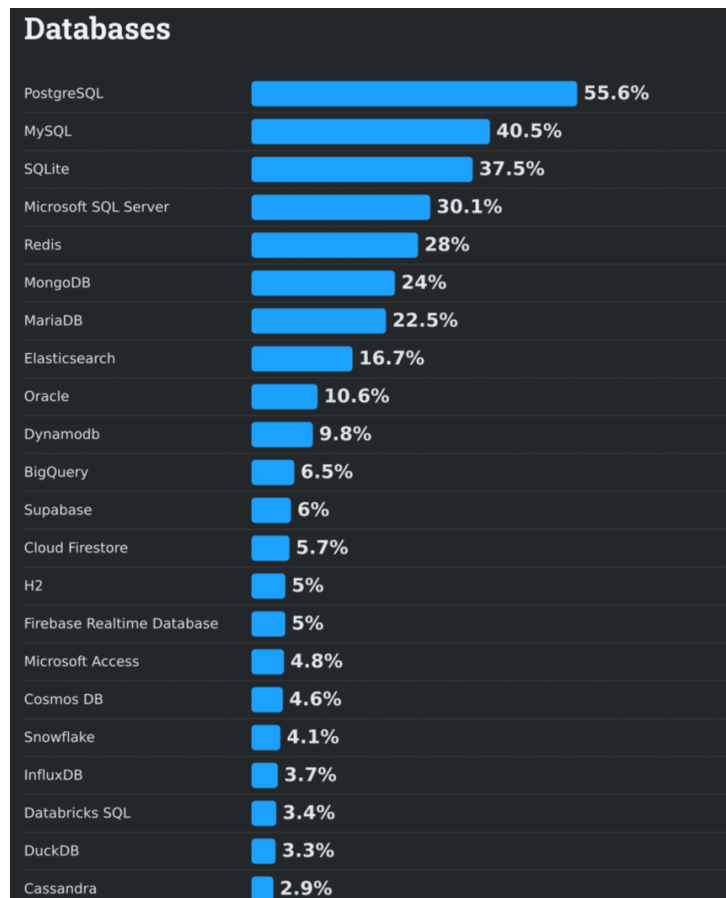


Figura 2.2: Encuesta sobre bases de datos. Fuente: Stack Overflow [1]

2.3.4. Despliegue

Durante el desarrollo local de una aplicación full-stack, es habitual ejecutar por separado las distintas partes del sistema. En el frontend, herramientas como Vite permiten iniciar rápidamente un servidor de desarrollo para visualizar los cambios de la interfaz durante el desarrollo. En el backend y la base de datos, es habitual utilizar contenedores para ejecutar servicios de forma aislada y reproducible, evitando problemas al cambiar entre equipos.

En aplicaciones formadas por varios servicios, herramientas como Docker y Docker Compose facilitan la ejecución de las distintas partes de la aplicación durante el desarrollo. Esto simplifica la puesta en marcha del sistema completo en local y permite gestionar de forma separada servicios como el backend y la base de datos [16].

Para publicar una aplicación en un entorno accesible desde internet, exis-

ten distintas alternativas. En el caso del frontend, plataformas como Vercel o Netlify están orientadas a simplificar el despliegue de aplicaciones web, conectándose directamente al repositorio de GitHub. Para el backend, existen plataformas como Render o Railway que permiten desplegar servicios web o API sin tener que gestionar manualmente la infraestructura.

Además de las opciones sencillas, existen plataformas cloud más completas y profesionales, como Amazon Web Services, Microsoft Azure o Google Cloud. Estas ofrecen un mayor número de servicios y permiten desplegar todas las partes de una aplicación. No obstante, su configuración es más avanzada y requiere un mayor conocimiento de infraestructuras.

En el despliegue de una aplicación, también es importante gestionar correctamente las claves y credenciales del sistema. Datos como la clave de la API o la contraseña de la base de datos no deben escribirse directamente en el código. Durante el desarrollo local, esta información suele guardarse en un archivo `.env`, que contiene variables de entorno utilizadas por la aplicación. En producción, estas variables se configuran directamente desde la plataforma de despliegue, como Vercel o Render. De esta forma, se evita exponer información sensible.

Nota: En los 2 últimos puntos, otra vez lo mismo; podría poner fuentes de vite, docker, docker compose, vercel, render, railway, aws... No sé si poner una fuente web de absolutamente todo lo que mencione o solo de lo más importante.

2.4. Limitaciones de las soluciones existentes

Las herramientas analizadas anteriormente presentan funcionalidades muy útiles para los jugadores de LoL, pero suelen estar enfocadas en una necesidad concreta. Algunas aplicaciones se centran principalmente en la consulta de perfiles e historiales de partidas, mientras que otras están más enfocadas en builds, estadísticas de campeones o seguimiento de jugadores profesionales. Esto provoca que el usuario tenga que consultar varias herramientas diferentes para obtener la información que necesita.

Otra limitación muy común es la sobrecarga de información. Muchas de estas plataformas muestran una gran cantidad de datos, estadísticas, recomendaciones, comparativas y elementos visuales en una misma interfaz. Aunque esto puede ser útil para jugadores muy avanzados, también puede dificultar la búsqueda de información concreta para usuarios menos experi-

mentados.

Recientemente, las herramientas de historiales o análisis de partidas han añadido funciones con inteligencia artificial que asignan una puntuación al rendimiento del jugador en cada partida. Sin embargo, esta puntuación no suele basarse en criterios claros y, muchas veces, no concuerda con lo acontecido durante la partida. Al ser un entorno con mucho tráfico y una fuerte competencia entre plataformas, estas herramientas tienden a incorporar funcionalidades llamativas con el fin de diferenciarse y atraer más usuarios. No obstante, estas funciones no llegan a satisfacer una necesidad real del usuario, sino que funcionan más como añadidos para atraer público que como herramientas realmente útiles.

En algunos casos, el problema no es solo la cantidad de información, sino también la forma en la que se presenta. Al consultar una build, algunas plataformas muestran varias combinaciones posibles de objetos en lugar de ofrecer una recomendación más directa. Aunque esta variedad puede ser útil para jugadores experimentados, no siempre resulta adecuada para los usuarios que buscan una respuesta rápida y sencilla.

Estas limitaciones muestran que, aunque existen herramientas muy completas en el ecosistema de LoL, no siempre están preparadas para todo tipo de usuarios. Por ello, todavía existe espacio para nuevas aplicaciones más sencillas, directas y centradas en nuevos casos de uso.

2.5. Aportación de este trabajo

La aportación principal de este trabajo consiste en el desarrollo de una aplicación web full-stack orientada al análisis y consulta de datos de LoL. Para ello, se integran en una misma herramienta varias funcionalidades habituales en este tipo de aplicaciones, como la búsqueda de jugadores, la consulta de información básica del perfil, el historial de partidas, los detalles de cada partida, el estado de la partida actual y el seguimiento de builds de jugadores profesionales.

Este trabajo no busca sustituir a plataformas consolidadas como OP.GG o Blitz.gg, sino ofrecer una alternativa más sencilla y centralizada. La idea principal es reunir en una misma aplicación la información básica que un jugador puede necesitar, evitando que tenga que acudir a varias herramientas para satisfacer sus necesidades.

Además, el trabajo aporta una solución técnica completa mediante el diseño e implementación de una arquitectura formada por frontend, backend

y base de datos. El frontend se encarga de la interacción con el usuario, el backend centraliza la lógica de negocio y la comunicación con la API de Riot, y la base de datos permite almacenar la información consultada. Esta organización facilita el mantenimiento del sistema y permite reducir llamadas repetidas a la API reutilizando los datos ya almacenados.

También se puede considerar una aportación metodológica, ya que el proyecto se desarrolla de forma iterativa e incremental. En lugar de intentar construir desde el inicio una aplicación completa con todas las funcionalidades posibles, se parte de un MVP sobre el que se van añadiendo nuevas características. Este enfoque permite validar progresivamente cada incremento y dejar preparada una base sobre la que incorporar futuras funcionalidades.

Nota: Esto lo he puesto porque me lo dijiste en el correo del índice, pero me suena a paja. ¿Cómo lo ves?.

Actualización tras terminar el trabajo: Ahora veo que también se puede solapar un poco con el "Aportaciones e impacto de Easy-Rift" de las conclusiones. Puedo quitar los párrafos 3 y 4 o directamente quitar el apartado entero, tú mandas.

Por todo esto, este trabajo aporta una aplicación centralizada con un enfoque más sencillo, accesible y adaptado a los usuarios menos experimentados. No se basa únicamente en la cantidad de datos mostrados, sino en la forma de organizarlos y presentarlos para facilitar la experiencia del usuario.

Capítulo 3

Marco teórico

En este capítulo se presentan los conceptos necesarios para comprender el desarrollo técnico del proyecto. No se pretende realizar un estudio exhaustivo del videojuego, sino introducir el vocabulario principal relacionado con LoL y con la obtención de datos desde servicios externos. El objetivo es proporcionar una base conceptual que facilite la lectura de los siguientes capítulos.

3.1. Conceptos básicos de League of Legends

LoL es un videojuego competitivo del género MOBA en el que dos equipos formados por cinco jugadores compiten entre sí. Cada jugador selecciona un campeón y ocupa una posición determinada dentro de un mapa. Durante la partida, los jugadores deben tomar decisiones relacionadas con su posición en el mapa, la coordinación del equipo, la compra de objetos o el orden de mejora de sus habilidades.

Gran parte de los conocimientos necesarios para tomar buenas decisiones a lo largo de la partida se obtienen con la experiencia. No obstante, algunas decisiones, como qué campeón escoger, qué objetos comprar o el orden de mejora de sus habilidades, pueden saberse rápidamente analizando los datos disponibles en la API de Riot.

Para satisfacer la necesidad del jugador de tener una fuente fiable de información a la que acudir, se comenzaron a desarrollar aplicaciones de análisis de datos de videojuegos. Estas aplicaciones utilizan información procedente de partidas, jugadores, campeones y otros elementos del juego.

Por todo esto, para entender el funcionamiento de este trabajo, es necesario introducir algunos conceptos del videojuego:

- **Campeón:** Personaje jugable seleccionado por cada jugador al inicio de una partida. Cada campeón tiene habilidades y características diferentes. Actualmente, el videojuego cuenta con 173 campeones diferentes.

Nota: Para saber el número de campeones es meterte al videojuego y mirarlo, cómo pongo eso de fuente? porque el videojuego no es ninguna web ni nada para citar

- **Rol o posición:** Función de un jugador dentro de un equipo. En LoL hay cinco posiciones diferentes: Top, Jungla, Mid, Bot y Support. La API de Riot trata las posiciones como TOP, JUNGLE, MIDDLE, BOTTOM y UTILITY respectivamente. Cada una define las responsabilidades de un jugador dentro del equipo y la zona del mapa de la que se encarga.
- **Build:** Conjunto de objetos comprados por un jugador durante la partida. Una build está formada por un máximo de 6 objetos. Cada objeto tiene un coste y otorga estadísticas diferentes al campeón que lo posee. Los objetos elegidos y el orden en el que se compren son una de las decisiones más importantes a la hora de ganar una partida. Hoy en día, el LoL cuenta con más de 200 objetos diferentes.
- **Parche:** Versión del juego publicada por Riot Games. Cada parche puede introducir cambios en campeones, objetos o sistemas dentro del videojuego. Suele haber un parche cada dos semanas con el fin de equilibrar los diferentes campeones para que ninguno sobresalga por encima del resto.
- **Meta:** Conjunto de campeones, objetos o estrategias más eficientes en el momento actual del juego. El meta cambia constantemente debido a los numerosos parches que Riot realiza al LoL, por lo que, para tener un buen rendimiento, es esencial conocer el meta en todo momento.
- **Cola:** Aunque la cola o modo de juego principal de LoL siempre ha sido la Grieta del Invocador, el videojuego dispone de modos de juego extra donde cambian algunas características, como el número de jugadores, el mapa o la selección del campeón. El competitivo se basa en el modo principal; no obstante, la API de Riot muestra datos de todo tipo de partidas. Aunque las recomendaciones están basadas únicamente en las partidas del modo principal, en el historial de un jugador se verán reflejadas partidas de cualquier modo.

- **Tasa de victorias y tasa de selección:** Estas métricas permiten analizar el rendimiento y la frecuencia de uso de campeones y objetos. Su combinación ayuda a identificar cuáles ofrecen mejores resultados y tienen mayor presencia en el meta actual.
- **Jugador profesional:** Jugador que compite representando a un equipo en ligas y torneos. El entorno competitivo de LoL se divide en diferentes ligas regionales, cuyos mejores equipos participan posteriormente en competiciones internacionales. Debido a que estos jugadores compiten al máximo nivel y deben adaptarse a los cambios del videojuego, sus partidas son una fuente de información útil para conocer los campeones, objetos y estrategias utilizados en el entorno competitivo [17].

Teniendo estos conceptos en cuenta, una aplicación de análisis de datos de LoL debe proporcionar la información necesaria para conocer el meta. Por ello, este tipo de aplicaciones muestran historiales con detalles sobre las partidas jugadas, estadísticas por campeón para conocer qué campeones tienen mayor tasa de victorias o son más jugados en el momento e incluso builds recomendadas para estos campeones.

Nota: Puedo entrar en más detalle, pero creo que con esto es suficiente. Tú me dices; tu opinión es muy importante aquí porque es la de alguien que jamás ha jugado al juego, igual que el tribunal

3.2. Obtención y tratamiento de datos

3.2.1. API de Riot

Una API es un conjunto de reglas o protocolos que permiten que diferentes aplicaciones se comuniquen entre sí. En este contexto, una API externa es un servicio proporcionado por un tercero que permite consultar o enviar información desde otra aplicación [18].

Las diferentes operaciones ofrecidas por una API se encuentran disponibles mediante endpoints. Un endpoint es una dirección concreta, normalmente una URL, a la que se envía una petición para acceder a un recurso o ejecutar una operación determinada [19].

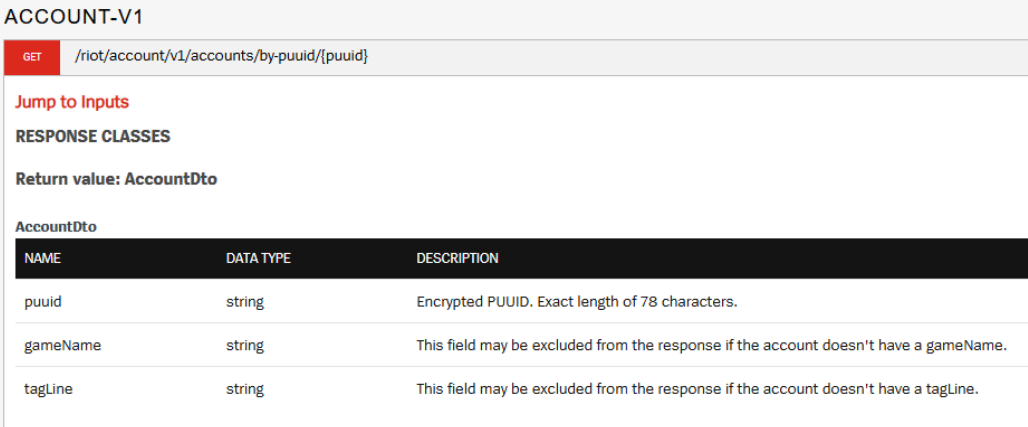
En este proyecto, la API externa utilizada es la API de Riot, ya que proporciona todos los datos necesarios para obtener información de jugadores, partidas y otros elementos del videojuego. Esta API sigue un modelo

REST, uno de los más populares actualmente. Las API REST se comunican haciendo uso del protocolo HTTP.

En la aplicación desarrollada en este trabajo, el frontend no se comunica directamente con la API de Riot. Las peticiones se realizan al backend, que actúa como intermediario entre el usuario y la API de Riot.

Cuando se realiza una petición a la API de Riot, esta devuelve una respuesta con información estructurada en formato JSON. Este formato permite representar los datos siguiendo una estructura de pares clave-valor.

Riot representa estas respuestas mediante DTO, objetos utilizados para transportar datos. Como se muestra en la Figura 3.1, cada llamada a la API devuelve un objeto formado por diferentes campos, dependiendo de la consulta realizada [12].



ACCOUNT-V1

GET /riot/account/v1/accounts/by-puuid/{puuid}

[Jump to Inputs](#)

RESPONSE CLASSES

Return value: AccountDto

AccountDto

NAME	DATA TYPE	DESCRIPTION
puuid	string	Encrypted PUUID. Exact length of 78 characters.
gameName	string	This field may be excluded from the response if the account doesn't have a gameName.
tagLine	string	This field may be excluded from the response if the account doesn't have a tagLine.

Figura 3.1: DTO devuelto por la API de Riot. Fuente: Riot Games [2]

3.2.2. Riot ID

En LoL, los jugadores se identifican mediante el Riot ID. Este identificador está formado por dos partes: el nombre de jugador y una etiqueta. Un ejemplo de Riot ID sería `iFran28#EUW`, donde `iFran28` es el nombre del jugador y `EUW` la etiqueta. Esto permite que puedan existir nombres repetidos, siempre que usen una etiqueta diferente [20].

Conocer esta estructura de nombrado es importante ya que la API de Riot utiliza el nombre y la etiqueta para localizar una cuenta. A partir de estos datos, la API devuelve su propio identificador interno, el *Player Universally Unique Identifier* (PUUID), que se utiliza posteriormente para realizar consultas relacionadas con el jugador en cuestión [2].

A la hora de buscar un jugador, además del Riot ID, se debe incluir la plataforma en la que el jugador está registrado.

3.2.3. Plataformas y regiones

La API de Riot no utiliza una única dirección para todas las peticiones. Para realizar una llamada a la API correctamente, es necesario seleccionar el host adecuado en función del endpoint que se quiere consultar. Riot utiliza valores de enrutamiento que pueden corresponder a una plataforma o a una región.

Las plataformas representan los servidores del videojuego, como EUW1, para el oeste de Europa, o KR, para Corea del Sur. Por otro lado, las regiones agrupan varias plataformas, como EUROPE, AMERICAS, ASIA o SEA.

Cada endpoint de la API de Riot muestra la plataforma o región sobre la que se debe hacer la petición. Como se puede ver en la Figura 3.2, en el caso del endpoint MATCH-V5, la documentación oficial muestra que para las llamadas realizadas a dicho endpoint, se deben utilizar las regiones [12].

The AMERICAS routing value serves NA, BR, LAN and LAS. The ASIA routing value serves KR and JP. The EUROPE routing value serves EUNE, EUW, MEL, TR and RU. The SEA routing value serves OCE, SG2, TW2 and VN2.	
MATCH-V5	
GET	/lol/match/v5/matches/by-puuid/{puuid}/ids Get a list of match ids by puuid
GET	/lol/match/v5/matches/by-puuid/{puuid}/replays Get player replays
GET	/lol/match/v5/matches/{matchid} Get a match by match id
GET	/lol/match/v5/matches/{matchid}/timeline Get a match timeline by match id

Figura 3.2: Endpoint MATCH-V5. Fuente: Riot Games [2]

Nota: Esto es raro, dentro de cada endpoint (hay muchísimos), te dice si debes usar europe/asia/america/sea o el servidor concreto EUW1/KR/NA1... En algunas te dice que puedes usar cualquiera y funciona, y en otras te dice que debes una usar región o servidor concreto.

Al final la region es una forma de agrupar servidores. He puesto el ejemplo de match-v5 porque al final son partidas y es el más básico pero se ve enana la aclaración oficial arriba (No sé si quitarla). También hay tablas con todos los servidores y regiones, pero veo excesivo poner eso, aunque no me cuesta nada.

He usado también la de match porque arriba te dice a qué región corresponde cada plataforma/servidor concreto.

Por tanto, una petición al endpoint MATCH-V5 para consultar una partida

de un jugador perteneciente al servidor europeo tendría como base la siguiente URL:

```
https://europe.api.riotgames.com/lol/match/v5/matches/matchId
```

Esto implica que la aplicación debe distinguir entre la plataforma seleccionada por el usuario, por ejemplo EUW, y la región asociada que se debe utilizar en determinados endpoints, en este caso EUROPE. No obstante, como cada plataforma pertenece a una región concreta, la aplicación puede obtener esta región internamente. Por ello, el usuario solo debe indicar la plataforma a la que pertenece el jugador que desea buscar.

3.2.4. Partidas y participaciones

La API de Riot devuelve la información de una partida en un objeto llamado `MatchDto`. Este objeto contiene, por un lado, metadatos de la partida, como su identificador, y por otro, información general sobre la misma, como la duración, el modo de juego o la versión del parche.

Además de estos datos generales, una partida tiene información de todos los jugadores que han participado en ella. Riot representa esta información mediante participaciones. En una partida estándar hay diez participaciones, una por cada jugador.

Cada participación guarda los datos de un jugador dentro de esa partida. Entre ellos se encuentran datos relevantes como el campeón utilizado, el rol, las estadísticas principales y los objetos comprados. Por este motivo, las participaciones son uno de los elementos más importantes para este trabajo.

Capítulo 4

Análisis del problema

En este capítulo se analiza el problema que pretende resolver la aplicación. Para ello, se establecen los requisitos del sistema y se describen los componentes que tendrá la aplicación. Además, se presenta la arquitectura general del sistema junto con los flujos de información entre el usuario, los componentes y los servicios externos.

Nota: en el guión que me diste ponía "Análisis del problema y metodología empleada"; no sé cómo poner lo de metodología porque en el apartado 6 va de eso y en la introducción ya lo nombré, se me quedaba muy repetitivo.

4.1. Planteamiento del problema

El problema que aborda este trabajo surge de la complejidad de LoL y de la necesidad que tienen los jugadores de consultar información actualizada de forma rápida y sencilla. LoL es un videojuego competitivo con una gran cantidad de variables: campeones, roles, objetos, parches y estrategias de juego diferentes. Esta complejidad hace que los jugadores necesiten mantenerse actualizados para tomar las mejores decisiones dentro de una partida.

Además de la necesidad de información constante, la dificultad del videojuego genera una barrera de entrada elevada para los nuevos jugadores. Riot Games ha reconocido en varias publicaciones oficiales que el videojuego presenta una curva de aprendizaje muy alta para los nuevos jugadores. Con el fin de reducirla, Riot ha tomado medidas como facilitar el videojuego y crear controles más amigables para nuevos usuarios [21][22].

Nota: Las 2 referencias seguidas no sé si está bien así, también deja ponerlo en estilo [1, 2] pero a mí me parece bastante feo.

Actualmente, existen distintas herramientas que permiten obtener datos sobre perfiles, partidas, campeones o builds. Sin embargo, muchas de ellas están muy especializadas o muestran una cantidad excesiva de información en pantalla. Esto puede ser útil para jugadores experimentados, pero dificulta la experiencia de los nuevos usuarios, que buscan una respuesta rápida y sencilla.

Por todo esto, el problema no es la ausencia de datos, sino la dificultad de centralizarlos, organizarlos y presentarlos de forma clara para que el usuario novato pueda consultarlos fácilmente.

Desde el punto de vista técnico, el problema consiste en obtener la información desde una fuente externa, procesarla, almacenarla y mostrarla de forma comprensible al usuario. La API de Riot proporciona los datos necesarios, pero se encuentran repartidos en distintos endpoints y se devuelven en estructuras que no están pensadas para el usuario final. Esto hace necesario plantear una solución que actúe como intermediaria entre los datos disponibles en la API de Riot y el usuario que desea consultarlos.

Durante el desarrollo y la evaluación de EasyRift se utiliza una *Personal API Key*, limitada a cien peticiones cada dos minutos. Esta restricción condiciona el funcionamiento de la aplicación, ya que obliga a controlar las llamadas realizadas y evitar que los procesos automáticos impidan las consultas de los usuarios. Por tanto, el sistema debe obtener una cantidad suficiente de datos sin superar el límite establecido por Riot Games [12].

La solución propuesta para este problema es el desarrollo de una aplicación web que consulte, procese y organice los datos obtenidos de la API de Riot, para que el usuario pueda acceder a la información de forma centralizada, clara y sencilla.

4.2. Requisitos del sistema

A partir del problema planteado, se definen los requisitos que debe cumplir la aplicación. Estos requisitos se dividen en requisitos funcionales, que describen qué debe hacer el sistema, requisitos no funcionales, que definen cómo debe operar el sistema, y unos requisitos funcionales de ampliación, con mejoras funcionales para ampliar el sistema en incrementos posteriores.

A continuación, se muestran los requisitos definidos para el sistema. Estos

se identifican como funcionales (RF), no funcionales (RNF) y funcionales de ampliación (RFA).

ID	Requisito	Descripción
RF1	Búsqueda de perfil	El sistema debe permitir buscar un jugador mediante su Riot ID y la plataforma correspondiente.
RF2	Consulta de perfil	El sistema debe mostrar la información básica del jugador buscado.
RF3	Historial de partidas	El sistema debe permitir consultar el historial de partidas recientes de un jugador.
RF4	Detalle de partida	El sistema debe permitir consultar los detalles de una partida concreta del historial.
RF5	Estado de la partida actual	El sistema debe indicar si el jugador se encuentra actualmente en partida y mostrar información básica de la misma.
RF6	Campeones	El sistema debe permitir ver todos los campeones disponibles.
RF7	Builds de profesionales	El sistema debe permitir consultar builds recientes utilizadas por jugadores profesionales para un campeón concreto.
RF8	Información de objetos	El sistema debe mostrar la información básica de los objetos al interactuar con ellos.
RF9	Información actualizada	El sistema debe permitir actualizar la información de jugadores y partidas consultando la API de Riot.

Tabla 4.1: Requisitos funcionales.

Nota: Luis nos hizo en su día hacer también requisitos del sistema. Ej: el sistema operativo será windows. Yo no lo he puesto porque lo considero paja, pero podría ponerlos.

ID	Requisito	Descripción
RNF1	Usabilidad y accesibilidad	La interfaz debe ser clara y sencilla, permitiendo que usuarios con distinto nivel de conocimiento puedan consultar la información sin dificultad.
RNF2	Persistencia de datos	El sistema debe almacenar la información para reutilizar datos ya consultados y reducir el número de llamadas a la API externa.
RNF3	Gestión de errores	El sistema debe gestionar los errores derivados de la API externa.
RNF4	Separación de responsabilidades	El sistema debe separar la interfaz de usuario, la lógica de negocio y la persistencia de datos.
RNF5	Despliegue local	El sistema debe poder ejecutarse en un entorno local separando sus componentes principales.
RNF6	Adaptabilidad	La interfaz debe adaptarse a diferentes tamaños de pantalla y permitir su uso tanto en ordenadores como en dispositivos móviles.
RNF7	Despliegue en producción	El sistema debe desplegarse mediante servicios de alojamiento externo y ser accesible a través de un dominio propio.

Tabla 4.2: Requisitos no funcionales.

ID	Requisito	Descripción
RFA1	Estadísticas de campeones	El sistema debe mostrar estadísticas relacionadas con campeones.
RFA2	Estadísticas de jugadores	El sistema debe mostrar estadísticas relacionadas con el rendimiento de los jugadores.

Continúa en la siguiente página

ID	Requisito		Descripción
RFA3	Actualización automática	au-	El sistema debe automatizar la actualización de información mediante tareas programadas que se ejecuten periódicamente.
RFA4	Recomendación de builds	de	El sistema debe mostrar una build recomendada para cada combinación de campeón y rol a partir de las partidas clasificatorias del parche actual.
RFA5	Rangos clasificados	clasificato-	El sistema debe mostrar los rangos clasificatorios actuales del jugador en las colas Solo/Duo y Flex.
RFA6	Clasificación de campeones	de	El sistema debe clasificar los campeones mediante tiers a partir de las partidas clasificatorias del parche actual.
RFA7	Consulta de profesionales	de profe-	El sistema debe permitir consultar los jugadores profesionales agrupados por equipos y acceder a sus perfiles profesionales y builds recientes.
RFA8	Selección por rol		El sistema debe permitir seleccionar un rol en la consulta de un campeón y adaptar la build recomendada y las builds profesionales recientes al rol seleccionado.

Tabla 4.3: Requisitos funcionales de ampliación.

Nota: en su día, hice los objetivos de ampliación, que se traducen aquí como requisitos funcionales de ampliación. El tema es que puse como objetivo de ampliación "Preparar el paso a producción", básicamente que la web sea accesible en internet con un dominio, un hosting... La cosa es; eso es un requisito no funcional. ¿Lo meterías en no funcionales aunque sea de ampliación, lo ignorarías, o harías una tabla de requisitos funcionales y otra de no funcionales sin separarlos en "de ampliación"?

4.3. Arquitectura general y componentes del sistema

La aplicación se basa en una arquitectura cliente-servidor formada por varios componentes principales. En la Figura 4.1 se muestra el diagrama de componentes de alto nivel del sistema.

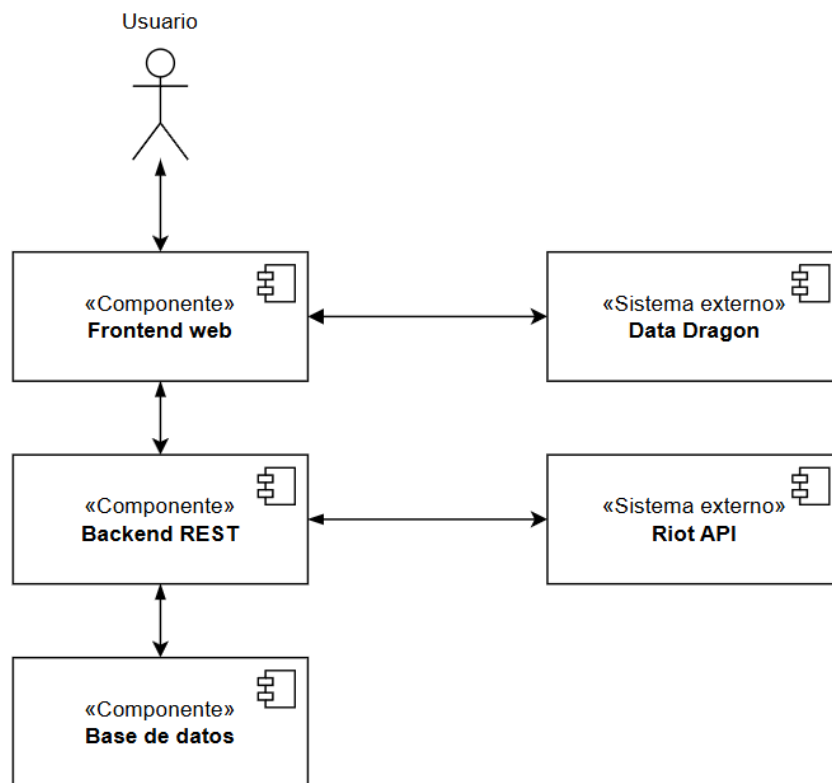


Figura 4.1: Diagrama de componentes de alto nivel

El usuario interactúa desde un navegador web, donde se ejecuta el frontend. El frontend, a su vez, no interactúa directamente con la API de Riot, sino que se comunica con el backend mediante peticiones HTTP y carga los datos estáticos proporcionados por Data Dragon.

El backend es el componente central del sistema. Su función es recibir las peticiones del frontend, aplicar la lógica de negocio necesaria, consultar la base de datos en busca de información ya almacenada y comunicarse con la API de Riot para obtener o actualizar datos cuando sea necesario.

La base de datos se utiliza para almacenar la información que el sistema puede reutilizar. Se guardan los datos relacionados con jugadores, partidas, participaciones, campeones, builds y jugadores profesionales.

Esta separación en componentes permite que cada parte del sistema tenga una responsabilidad concreta. El frontend se encarga de la interacción con el usuario, el backend de la lógica de negocio y el acceso a los datos, la base de datos de almacenar la información y los servicios externos de proporcionar los datos y recursos necesarios para el funcionamiento de la aplicación.

4.4. Flujo de información

Una vez se ha definido la arquitectura general, se van a describir los principales flujos de información del sistema. En este apartado no se pretende explicar la implementación o diseño de la aplicación, sino representar de forma general cómo se obtiene, organiza y muestra la información.

El primer flujo corresponde a la consulta de información de un jugador, ya que es una de las funciones principales de la aplicación. El usuario introduce el Riot ID y la plataforma correspondiente desde el frontend. A partir de estos datos, la solicitud se envía al backend, que se encarga de procesarla y comprobar si existe información almacenada en la base de datos previamente.

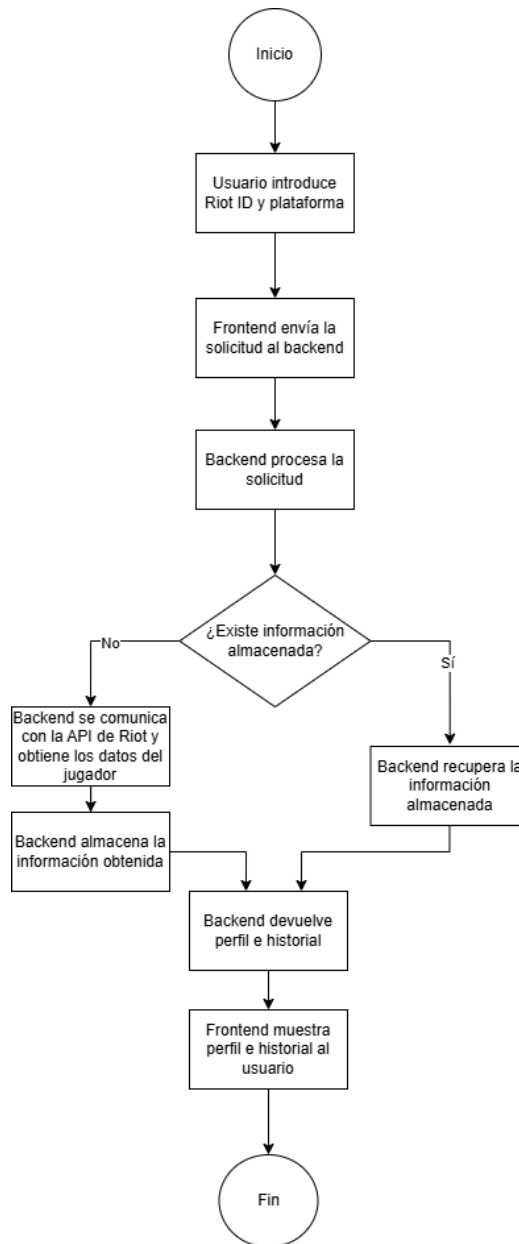


Figura 4.2: Diagrama de flujo. Perfil de usuario

Como se muestra en la Figura 4.2, si la información ya se encuentra almacenada, el backend la recupera y devuelve al frontend. En caso contrario, consulta a la API de Riot para obtener los datos del jugador, almacena la información obtenida y devuelve al frontend el perfil junto con el historial de partidas, que se mostrará al usuario.

El segundo flujo representa la consulta de builds recientes de jugadores profesionales. En este caso el usuario selecciona un campeón desde el frontend y se vuelve a enviar la solicitud al backend. A diferencia del flujo anterior, la consulta de builds se realiza sobre las partidas ya almacenadas en la base de datos previamente.

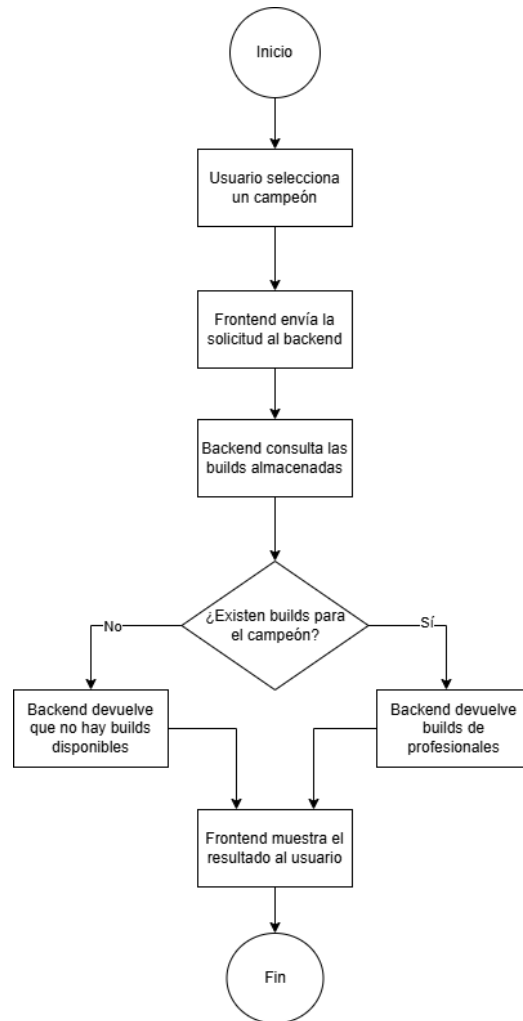


Figura 4.3: Diagrama de flujo. Builds de profesionales

Como se muestra en la Figura 4.3, el backend comprueba si existen builds disponibles para el campeón seleccionado. Si existen, devuelve las builds de los profesionales al frontend para que se muestren al usuario. Si no hay builds disponibles, el backend devuelve una respuesta vacía y el frontend muestra dicho resultado.

Estos dos flujos representan las dos funciones principales de la aplicación: consultar información de un jugador concreto y consultar información relacionada con campeones y jugadores profesionales. En el primer caso, la información se obtiene directamente desde la API de Riot y puede almacenarse para futuras consultas. En el segundo caso, las builds profesionales se obtienen a partir de información ya almacenada y procesada por el sistema.

Nota: ¿Debo poner algo de viabilidad del proyecto, planificación de horas...? En el correo no me lo mencionaste, pero otros profesores lo piden. A mí me parece un poco absurdo porque al aprender tecnologías nuevas y hacer cosas que nunca hice, es imposible que exista una planificación buena, sería inventar (Además, que nunca supe hasta donde iba a hacer, porque esto es infinito). El recuento de horas lo seguí un tiempo hasta llegar a casi 200 horas, últimamente no lo he apuntado pero me puedo hacer una idea de las horas si hace falta.

Capítulo 5

Diseño del sistema

En este capítulo se presenta el diseño de EasyRift a partir de los requisitos definidos durante el análisis. Se describe la evolución funcional de la aplicación durante los tres incrementos, las tecnologías seleccionadas y el diseño de la base de datos, el backend, la API REST y el frontend.

5.1. Diseño funcional e incremental

La aplicación se lleva a cabo siguiendo un desarrollo incremental. Cada incremento cuenta con sus propias fases de análisis, diseño, implementación y pruebas, y añade nuevas funcionalidades sobre la versión obtenida en el incremento anterior. Este planteamiento permite disponer de una aplicación funcional desde las primeras etapas y ampliar progresivamente sus funciones sin perder las ya desarrolladas en incrementos previos.

La Figura 5.1 muestra el diagrama final de casos de uso, en el que se encuentran todas las funcionalidades implementadas durante los tres incrementos realizados. Los colores de los casos de uso permiten identificar el incremento en el que se implementa cada caso de uso: el azul corresponde al primer incremento, el verde al segundo y el rojo al tercero.



Figura 5.1: Diagrama de casos de uso

5.1.1. Primer incremento: MVP

El primer incremento se plantea como un MVP. Su objetivo es establecer una primera versión funcional que incluya las operaciones básicas de consulta de información de jugadores, partidas y campeones.

La funcionalidad principal de este incremento es *Consultar perfil de juga-*

dor. El usuario introduce el Riot ID junto con la plataforma correspondiente, y la aplicación obtiene y muestra la información básica del jugador.

Desde el perfil se incluye el caso de uso *Consultar historial de partidas*, que muestra las partidas recientes almacenadas junto con información como el campeón utilizado, el resultado, el índice de asesinatos, muertes y asistencias (KDA), la posición, los hechizos de invocador y la build final.

El usuario también puede seleccionar una partida del historial para acceder al caso de uso *Consultar detalles de partida*. Este permite ver información detallada, como la duración, el tipo de cola, el parche en el que se jugó y los participantes de ambos equipos.

Otra de las funcionalidades básicas es *Consultar partida en curso*. La aplicación comprueba si el jugador se encuentra jugando en ese momento. Cuando existe una partida activa, se muestra el campeón utilizado, el modo de juego y el tiempo transcurrido.

Por último, el MVP también permite consultar el catálogo de campeones disponibles. Al seleccionar uno de ellos, se accede a su información y a las builds utilizadas recientemente por jugadores profesionales.

El primer incremento construye una aplicación funcional y utilizable, aunque está limitada a funciones básicas. Estas funcionalidades establecen la base sobre la que se añaden las estadísticas y recomendaciones de los incrementos siguientes.

5.1.2. Segundo incremento: estadísticas y automatización

El segundo incremento consiste en ampliar la información disponible y añadir análisis de los datos almacenados. Las estadísticas de cada jugador se calculan al sincronizar su perfil, mientras que las estadísticas globales de los campeones se actualizan periódicamente mediante procesos automáticos.

Dentro del perfil se añaden los casos de uso: *Consultar estadísticas de jugador* y *Consultar rangos del jugador*. Las estadísticas muestran el rendimiento de cada jugador según las partidas almacenadas a través del número total, victorias, derrotas, tasa de victorias, KDA y los promedios de asesinatos, muertes y asistencias. Junto con ellas, también se determina el mejor campeón del jugador mediante su frecuencia de uso y su tasa de victorias.

La consulta de rangos muestra la clasificación del jugador en las colas Solo/Duo y Flex, siempre que haya un rango disponible. Para cada clasificación

se muestran el rango, la división, los puntos de liga y la tasa de victorias, calculada a partir de sus victorias y derrotas.

En este incremento también se añaden las estadísticas de campeones. Al acceder a un campeón, se muestran las partidas registradas, victorias, derrotas, tasa de victorias, KDA y los valores medios de asesinatos, muertes y asistencias. Estos datos se calculan utilizando las participaciones almacenadas correspondientes a las partidas clasificatorias.

Además, se utilizan procesos automáticos encargados de actualizar periódicamente los datos de los jugadores profesionales y recalcular las estadísticas globales de los campeones. Estos procesos forman parte del funcionamiento interno de la aplicación, por lo que no requieren ninguna acción del usuario.

5.1.3. Tercer incremento: clasificación y recomendaciones

El tercer y último incremento utiliza los datos recopilados y procesados para generar clasificaciones y recomendaciones.

En primer lugar, se encuentra el caso de uso *Consultar clasificación de campeones*. La aplicación asigna a cada campeón un tier entre S y E mediante una puntuación que combina su tasa de victorias y su tasa de selección normalizada. Esta clasificación permite comparar el rendimiento general de los campeones durante el parche actual, para que el usuario pueda elegir campeones que ofrecen una mayor probabilidad de victoria.

También se añade una de las funciones principales del sistema: *Consultar build recomendada por rol*. Para cada combinación de campeón y rol que disponga de suficientes partidas almacenadas, se calcula una recomendación utilizando las builds completas registradas en partidas clasificatorias del parche actual.

El usuario puede seleccionar uno de los roles disponibles para consultar tanto la build recomendada como las builds recientes de profesionales para esa misma posición. La consulta inicial de builds profesionales del primer incremento se amplía igualmente para permitir su filtrado por rol.

Por último, se añade un apartado específico para jugadores profesionales que, hasta este incremento, se encontraban en la base de datos pero no eran visibles para el usuario. El caso de uso *Consultar profesional* permite acceder a su perfil profesional, que muestra su equipo, liga, rol y builds recientes.

Este apartado se separa del perfil normal del jugador porque muestra información relacionada con su actividad competitiva. No obstante, desde el perfil profesional se permite acceder al perfil normal asociado a su cuenta para consultar sus rangos, estadísticas e historial.

5.2. Tecnologías seleccionadas

A partir de los requisitos definidos y de la arquitectura general presentada anteriormente, se seleccionan las tecnologías necesarias para desarrollar los distintos componentes de EasyRift. La elección se realiza teniendo en cuenta la separación entre la interfaz de usuario, la lógica de negocio y la persistencia, además de la necesidad de consultar servicios externos y desplegar cada componente de forma independiente.

Una vez presentadas estas tecnologías en el estado del arte, este apartado se centra en justificar su elección y explicar la función concreta que desempeña cada una dentro de la aplicación. La Tabla 5.1 muestra las tecnologías seleccionadas y el componente del sistema en el que se utilizan.

Tecnología	Componente	Función
PostgreSQL	Base de datos	Almacenamiento de datos.
<i>Java Persistence API</i> (JPA)	Persistencia	Simplificación del acceso a la base de datos mediante repositorios y consultas sobre las entidades.
Hibernate	Persistencia	Implementación de JPA encargada de relacionar las entidades Java con las tablas de PostgreSQL.
Java 21	Backend	Lenguaje de programación utilizado en el backend.
Spring Boot	Backend	Desarrollo y ejecución de la API REST.
RestClient	Backend	Realización de peticiones HTTP a los diferentes servicios de Riot Games.
Maven	Backend	Gestión de dependencias, compilación y generación del ejecutable del backend.

Continúa en la siguiente página

Tecnología	Componente	Función
React	Frontend	Construcción de la interfaz web mediante páginas y componentes reutilizables.
TypeScript	Frontend	Incorporación de tipado en los datos, componentes y respuestas recibidas desde el backend.
React Router	Frontend	Gestión de la navegación entre las distintas páginas de la aplicación.
Vite	Frontend	Entorno de desarrollo y generación de los archivos estáticos utilizados en el despliegue.
Tailwind CSS	Frontend	Estilos y adaptación de la interfaz a distintos tamaños de pantalla.
API de Riot Games	Servicio externo	Obtención de datos sobre cuentas, jugadores, rangos y partidas.
Data Dragon	Servicio externo	Obtención de imágenes y datos estáticos relacionados con campeones, objetos, hechizos e iconos de perfil.
Docker	Despliegue	Contenerización del backend para proporcionar un entorno de ejecución reproducible.
Docker Compose	Despliegue	Ejecución conjunta del backend y PostgreSQL durante el desarrollo local.
Render	Despliegue	Alojamiento del backend y de la base de datos PostgreSQL.
Vercel	Despliegue	Alojamiento del frontend.

Tabla 5.1: Tecnologías utilizadas en EasyRift

5.2.1. Base de datos

Para almacenar la información, se utiliza PostgreSQL, un sistema gestor de bases de datos relacional (SGBD). La elección de un modelo relacional se debe a que los datos gestionados por la aplicación están altamente rela-

cionados. Una participación pertenece a una partida y relaciona al jugador con el campeón utilizado y su build. Los rangos y las estadísticas del jugador se asocian al jugador, mientras que las estadísticas y recomendaciones de los campeones se calculan a partir de las participaciones almacenadas. El modelo relacional permite representar estas dependencias mediante claves primarias y ajenas.

Entre los SGBD relacionales disponibles, se selecciona PostgreSQL por ser de código abierto, estar ampliamente utilizado y ofrecer una buena integración con las tecnologías del backend. El acceso a la base de datos se gestiona mediante JPA, cuyas anotaciones permiten definir las entidades y sus relaciones. A través de repositorios y consultas se pueden realizar las operaciones necesarias trabajando con objetos Java, mientras que Hibernate se encarga internamente de traducirlas a operaciones sobre PostgreSQL. De esta forma, no es necesario escribir manualmente la mayoría de las sentencias SQL.

5.2.2. Backend

El backend se desarrolla utilizando Java 21 como lenguaje y Spring Boot como framework. Se selecciona Spring Boot porque permite integrar en un mismo entorno la API REST, el acceso a PostgreSQL, la validación, las tareas programadas y el tratamiento de errores requeridos por la aplicación.

Para comunicarse con los servicios externos de Riot se utiliza RestClient, proporcionado por Spring. Este cliente permite realizar peticiones HTTP y convertir las respuestas JSON en objetos Java automáticamente.

La gestión de dependencias y la construcción del backend se realizan mediante Maven. Este permite declarar las librerías utilizadas por el proyecto, ejecutar la compilación y generar el archivo ejecutable de la aplicación.

5.2.3. Frontend

La interfaz web se desarrolla mediante React y TypeScript. React permite reutilizar elementos comunes de EasyRift, como las tarjetas de partidas, rangos, estadísticas, builds y equipos. TypeScript se utiliza para añadir tipo al frontend, incluyendo variables, funciones, estados, propiedades de los componentes y estructuras de datos.

La navegación se gestiona mediante React Router, que conecta el buscador con los perfiles de jugadores, el catálogo y el detalle de campeones, así

como el listado de profesionales y sus perfiles específicos. Vite se utiliza como entorno de desarrollo y para generar la versión estática del frontend.

Los estilos se implementan mediante Tailwind CSS. Aunque las herramientas de inteligencia artificial generativa permiten crear estilos CSS rápidamente, la generación de hojas de estilo extensas puede producir reglas duplicadas, nombres inconsistentes y hacerse difícil de mantener. Tailwind permite utilizar criterios visuales consistentes entre páginas y componentes, reducir la cantidad de CSS personalizado y adaptar la interfaz a diferentes tamaños de pantalla.

5.2.4. Servicios externos

La Riot Games API se utiliza como fuente oficial de los datos dinámicos de EasyRift. Data Dragon proporciona las imágenes y datos estáticos empleados en la interfaz. El frontend consulta su última versión disponible para construir las direcciones de los iconos de campeones, objetos, hechizos y perfiles. Además, en el backend se utilizan archivos JSON locales descargados del propio Data Dragon para inicializar el catálogo de campeones o identificar los objetos completos.

5.2.5. Despliegue

Durante el desarrollo local, Docker Compose permite iniciar el backend y PostgreSQL conjuntamente, manteniendo los datos mediante un volumen. El frontend se ejecuta por separado utilizando el servidor de desarrollo de Vite, lo que permite visualizar los cambios de la interfaz sin generar manualmente una nueva versión estática.

Para producción se diseña una arquitectura formada por tres servicios independientes. El frontend se aloja en Vercel, mientras que el backend y PostgreSQL se alojan en Render. Esta separación permite desplegar cada componente de forma independiente y asignarle los recursos que necesita en cada momento.

Las plataformas seleccionadas gestionan el acceso público, los certificados HTTPS y el enrutamiento necesario para acceder a los servicios. Por este motivo, no es necesario configurar manualmente un servidor web o un proxy inverso como Nginx.

Esta arquitectura también permite adaptar los recursos si aumenta el número de usuarios. Render permite ampliar la capacidad del backend o eje-

cutar varias instancias entre las que se distribuyen las peticiones. El número de instancias puede ajustarse automáticamente según el uso de CPU o memoria. PostgreSQL también permite aumentar su capacidad y almacenamiento, y utilizar réplicas y mecanismos de alta disponibilidad [23][24].

Se eligen Render y Vercel en lugar de una plataforma más general como Amazon Web Services (AWS) porque su configuración es más sencilla y requiere menos conocimientos de administración de infraestructura. Para las necesidades de este trabajo, permiten desplegar la aplicación de forma rápida y automática.

5.3. Diseño de la base de datos

La base de datos es una parte fundamental, ya que permite conservar la información obtenida de Riot para ser reutilizada en consultas posteriores. También proporciona los datos necesarios para calcular las estadísticas de jugadores y campeones, clasificar campeones y generar las recomendaciones de builds.

El modelo entidad-relación evoluciona con los incrementos de la aplicación. En cada incremento, tras analizar las nuevas funcionalidades y actualizar el diagrama de casos de uso, se revisa el modelo de datos y se incorporan las entidades y relaciones necesarias para representar la nueva información, manteniendo la coherencia con los elementos definidos anteriormente.

La Figura 5.2 muestra el diagrama entidad-relación correspondiente a la versión final de la aplicación.

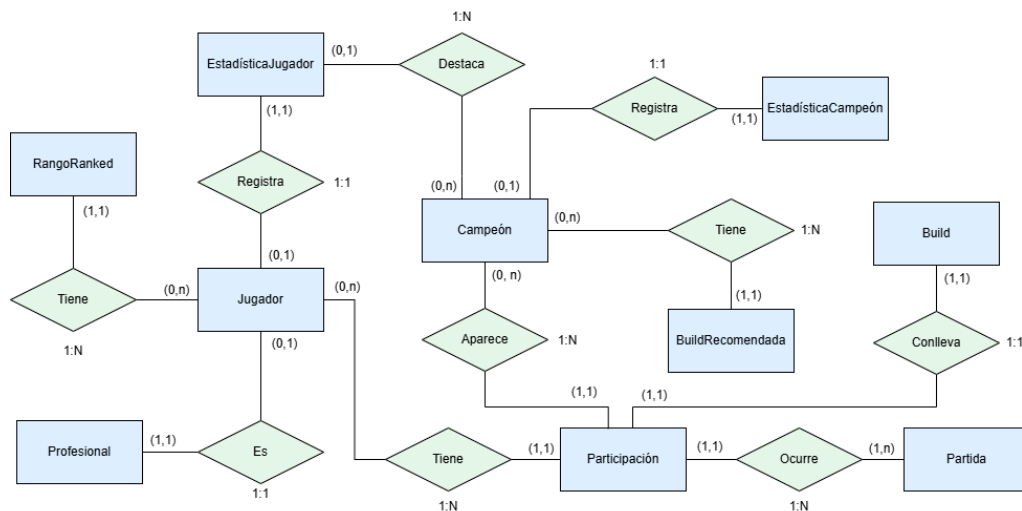


Figura 5.2: Diagrama entidad-relación final

Nota: Creo que explicarlo entero, poco a poco, es repetitivo, ya que el dibujo lo explica por sí mismo. El tfg de ejemplo de luis se pone a explicar los atributos de cada relación y el tipo de variable que son. Puedo hacerlo, no me cuesta nada, pero lo veo absurdo.

La información de cada partida se divide entre Partida y Participación. Partida almacena los datos comunes, como la fecha, la duración, el parche y el tipo de cola, mientras que Participación guarda los datos de cada uno de los jugadores, como el campeón utilizado, el resultado, el rol y el KDA. Cada participación se relaciona además con el jugador, el campeón y la build correspondientes. Esta separación evita repetir los datos generales de la partida para cada uno de sus diez participantes.

Aunque Build y Build recomendada contienen información similar sobre objetos y hechizos de invocador, representan conceptos diferentes. Build almacena los objetos y hechizos utilizados por un jugador en una participación concreta y, por tanto, se vincula a ella. En cambio, Build recomendada guarda el resultado calculado automáticamente para una combinación de campeón y rol. No pertenece a ninguna partida, jugador o participación concreta, aunque sí se relaciona con el campeón para el que se genera.

No se incluye el diagrama relacional porque las entidades y relaciones se definen mediante JPA y Hibernate se encarga de generar las tablas, las claves ajenas y, cuando es necesario, las tablas intermedias. La implementación de este proceso se explica posteriormente en el capítulo 6 mediante una entidad de ejemplo.

5.4. Diseño del backend

El backend realiza la lógica de negocio de la aplicación y actúa como intermediario entre el frontend, la base de datos y los servicios externos de Riot Games. Su diseño se centra en separar las responsabilidades de cada componente, evitar llamadas innecesarias a la API externa y reutilizar la información almacenada siempre que sea posible.

5.4.1. Diseño por capas

El backend se organiza en diferentes capas. Cada una se encarga de una función concreta:

- **Modelo:** contiene las entidades que representan la información almacenada en la base de datos.
- **Repositorios:** gestionan el acceso a la base de datos. Permiten realizar las operaciones *Create, Read, Update and Delete* (CRUD) de una forma sencilla.
- **Servicios:** contienen la lógica de negocio y coordinan el funcionamiento del backend.
- **Controladores:** reciben las peticiones HTTP realizadas por el frontend, utilizan los servicios correspondientes para procesarlas y construyen las respuestas mediante los DTO.
- **Capa de Riot:** agrupa la comunicación con los servicios externos de Riot Games. Incluye los clientes que realizan las peticiones HTTP, los DTO que representan las respuestas recibidas, el tratamiento de los errores de Riot y la construcción de las direcciones según la plataforma o región.

Además de estas capas, el backend incluye los siguientes componentes auxiliares:

- **DTO:** definen la información intercambiada entre el backend y el frontend.
- **Gestión de errores:** contiene las excepciones de la aplicación y su tratamiento.

- Cargadores: leen los archivos JSON locales utilizados para inicializar campeones, objetos y profesionales.
- Inicializadores: ejecutan las operaciones necesarias durante el arranque de la aplicación y se encargan de sincronizar el catálogo de campeones y la plantilla de profesionales.
- Planificadores: ejecutan periódicamente los procesos que no dependen de una petición del usuario.
- Configuración: contiene la configuración de *Cross-Origin Resource Sharing* (CORS) necesaria para permitir la comunicación con el frontend.

5.4.2. Diseño de la API REST

La comunicación entre el frontend y el backend se realiza mediante una API REST. Las peticiones utilizan el protocolo HTTP y las respuestas se devuelven en formato JSON. Los endpoints se agrupan según los elementos principales de la aplicación: jugadores, partidas, campeones y profesionales.

La Tabla 5.2 muestra los endpoints ofrecidos por la API.

Método	Ruta	Descripción
GET	/api/players/search	Busca un jugador mediante su plataforma, nombre de juego y etiqueta, y devuelve su perfil.
POST	/api/players/refresh	Actualiza la información de un jugador.
GET	/api/players/current-game	Consulta si un jugador se encuentra actualmente en una partida.
GET	/api/matches/{matchId}	Devuelve los detalles de una partida y sus participaciones.
GET	/api/champions	Devuelve el listado de campeones junto con su clasificación por tiers.

Continúa en la siguiente página

Método	Ruta	Descripción
GET	/api/champions/{championId}	Devuelve las estadísticas, las recomendaciones por rol y las builds profesionales recientes de un campeón.
GET	/api/professionals	Devuelve el listado de jugadores profesionales.
GET	/api/professionals/{puuid}	Devuelve la información y las builds recientes de un profesional.

Tabla 5.2: Endpoints de la API REST

La mayoría de los endpoints utilizan el método **GET**, ya que el usuario únicamente puede consultar información. La única excepción es la actualización de un jugador, que utiliza **POST** porque inicia una nueva sincronización y modifica los datos almacenados. No se hace uso de operaciones **PUT** o **DELETE**, puesto que la información se obtiene de Riot Games y es gestionada automáticamente por el backend.

Cuando una petición se completa correctamente, la API devuelve el código HTTP 200 junto con el DTO que contiene la información necesaria para la consulta realizada. Si se produce un error, se devuelve el código HTTP correspondiente y un DTO de error.

5.4.3. Sincronización de jugadores

Cuando se consulta un perfil, primero se busca al jugador en la base de datos mediante su plataforma, nombre y etiqueta. Si ya se encuentra almacenado en la base de datos, se devuelve la información disponible sin realizar llamadas innecesarias a Riot. Si no existe, se consulta la API externa para obtener su PUUID, que actúa como identificador del jugador.

Cuando el jugador se consulta por primera vez o se solicita una actualización, se sincronizan sus datos básicos, sus partidas recientes, sus rangos y sus estadísticas. Para reducir el número de peticiones a Riot se utiliza la fecha de la última sincronización y solo se solicitan partidas posteriores. Además, cada actualización se limita a un máximo de veinte partidas.

Cada partida se identifica mediante el identificador proporcionado por

Riot. Antes de almacenarla, se comprueba si ya existe, evitando duplicarla cuando aparece en el historial de diferentes jugadores. Al guardar una nueva partida, también se almacenan sus diez participaciones y los datos básicos de aquellos jugadores que todavía no se encuentran registrados.

El siguiente algoritmo muestra el proceso general seguido al consultar el perfil de un jugador:

Algoritmo 1: Consulta del perfil de un jugador

Entrada: Plataforma, nombre y etiqueta

Salida: Perfil del jugador

```
1  Buscar al jugador mediante su Riot ID
2  if el jugador existe then
3      Obtener su PUUID almacenado
4  end
5  else
6      Obtener su PUUID desde Riot
7  end
8  Bloquear el procesamiento del jugador mediante su PUUID
9  Buscar al jugador mediante su PUUID
10 if el jugador no existe then
11     Crear el jugador
12     Sincronizar sus datos
13 end
14 else if sus datos están incompletos then
15     Sincronizar sus datos
16 end
17 Liberar el bloqueo
18 return perfil del jugador
```

Para evitar que dos peticiones sincronicen simultáneamente al mismo jugador, se utiliza un cerrojo asociado a su PUUID. Este mecanismo garantiza la exclusión mutua, de forma que solamente una operación puede modificar los datos de un jugador en cada momento. Una vez finalizada la sincronización, el cerrojo se libera para permitir el procesamiento de nuevas peticiones a dicho jugador.

5.4.4. Procesos automáticos

Al iniciar el backend se preparan los datos básicos de la aplicación. El catálogo de campeones se carga desde un archivo JSON y se sincroniza con la base de datos. También se procesa la plantilla de profesionales para añadir nuevos jugadores y actualizar sus nombres, equipos, roles y ligas.

Después de la inicialización, se ejecutan dos procesos periódicos. El primero actualiza los datos y las partidas de los profesionales, comenzando por aquellos que llevan más tiempo sin sincronizarse. El segundo recalcula las estadísticas, los tiers y las builds recomendadas de los campeones a partir de la información almacenada.

Las estadísticas individuales no se actualizan mediante un proceso global. Se calculan cuando un jugador se consulta por primera vez o cuando se solicita la actualización de su perfil.

5.4.5. Cálculo de estadísticas y recomendaciones

Las estadísticas de cada jugador se calculan a partir de sus participaciones almacenadas. Entre los resultados obtenidos se encuentran el número de partidas, las victorias, la tasa de victorias, el KDA y los promedios de asesinatos, muertes y asistencias. También se determina su mejor campeón, teniendo en cuenta su rendimiento y frecuencia de uso.

Para las estadísticas globales de los campeones, solo se consideran las partidas pertenecientes a las colas clasificatorias y al parche más reciente almacenado. Esto evita mezclar datos de otros modos de juego o versiones anteriores del videojuego.

Para comparar las distintas alternativas se utiliza una puntuación inspirada en las funciones de aptitud, empleadas en los algoritmos genéticos. Estas funciones permiten combinar varios criterios en un único valor. En este caso no se implementa un algoritmo genético, sino que únicamente se aplica este concepto para valorar campeones y builds. La puntuación se calcula mediante la siguiente ecuación:

$$\text{Puntuación} = 0,6 \cdot \text{Tasa de victorias} + 0,4 \cdot \text{Tasa de selección normalizada} \quad (5.1)$$

Se asigna un peso del 60 % a la tasa de victorias y del 40 % a la tasa de selección normalizada. De esta forma, se concede mayor importancia al rendimiento, sin dejar de lado la frecuencia de uso. La puntuación se utiliza

para determinar el mejor campeón de un jugador, ordenar los campeones y asignarles un tier entre S y E, y seleccionar las builds recomendadas.

Las builds recomendadas se calculan de forma independiente para cada combinación de campeón y rol. Solo se consideran aquellas combinaciones que disponen de una cantidad mínima de diez partidas y que están formadas por seis objetos completos. Además, dos builds con los mismos objetos se consideran equivalentes aunque estos ocupen posiciones diferentes en el inventario.

Finalmente, las alternativas válidas se comparan mediante la puntuación calculada y se almacena la mejor build para cada combinación de campeón y rol.

5.5. Diseño del frontend

El frontend es el punto de interacción entre el usuario y las funciones proporcionadas por el backend. Su objetivo principal es ofrecer una interfaz sencilla que permita consultar e interpretar visualmente los datos obtenidos y procesados por la aplicación.

El núcleo del sistema es el backend, donde se realiza la comunicación con Riot, la persistencia de los datos y el cálculo de estadísticas, clasificaciones y recomendaciones. Por este motivo, el diseño del frontend prioriza la simplicidad y claridad de los resultados evitando interacciones complejas.

La interfaz se divide en tres áreas principales: consulta de jugadores, campeones y profesionales. Cada área organiza la información según las necesidades de sus casos de uso, manteniendo una estructura visual común mediante la reutilización de sus componentes.

5.5.1. Estructura y navegación

La cabecera se mantiene visible en todas las páginas y permite acceder directamente a los apartados: Buscador, Campeones y Profesionales. Esta navegación principal evita que el usuario tenga que regresar a la página inicial para cambiar de apartado. También se utiliza un pie de página constante para indicar que EasyRift es un proyecto educativo y que no está afiliado, respaldado ni patrocinado por Riot Games.

El buscador actúa como página inicial. Desde él, el usuario introduce la plataforma y el Riot ID del jugador que desea consultar. Si la búsqueda se

completa correctamente, se accede a su perfil, donde se muestran el estado de la partida actual, los rangos, las estadísticas y el historial de partidas.

Los detalles de una partida se muestran dentro del propio historial. Al seleccionar una partida, se despliega la información de sus participantes sin abandonar el perfil. Los nombres de los jugadores funcionan como enlaces a sus respectivos perfiles, facilitando la navegación entre jugadores de una misma partida.

El apartado de campeones muestra inicialmente la clasificación completa, organizada mediante diferentes tiers. También incluye un buscador que permite localizar un campeón concreto. Al seleccionar uno, se accede a una vista con sus estadísticas, la build recomendada y las builds recientes de profesionales. Un selector de roles permite cambiar la recomendación y las builds mostradas.

El apartado de profesionales presenta a los jugadores agrupados por equipos. Al seleccionar un profesional, se accede a una vista diferenciada del perfil normal, que muestra su equipo, liga, rol y builds recientes. Desde esta página se puede acceder también al perfil normal asociado a su cuenta. Además, los nombres de los profesionales mostrados en las builds recientes de un campeón permiten acceder directamente a su perfil profesional.

La Figura 5.3 muestra el árbol de navegación del frontend y las conexiones principales entre sus páginas.

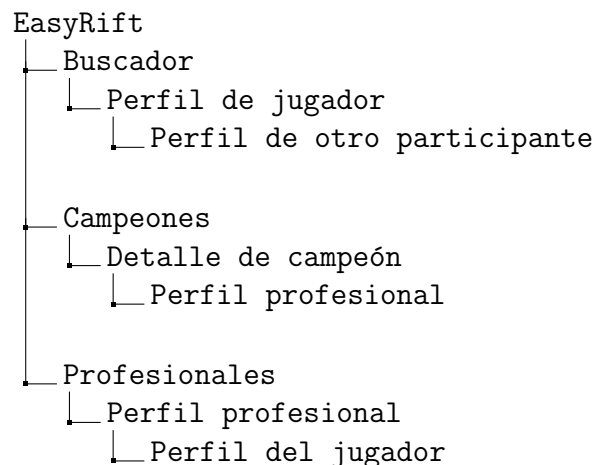


Figura 5.3: Estructura de navegación del frontend

La estructura mostrada se implementa como una SPA. React Router asocia cada ruta con la página correspondiente y permite navegar entre ellas sin recargar completamente la aplicación. Cuando es necesario, las rutas inclu-

yen identificadores como el del campeón o el PUUID del profesional, lo que permite cargar la información del elemento seleccionado.

5.5.2. Diseño basado en componentes

React permite construir la interfaz mediante componentes independientes que se combinan para formar las distintas páginas. Siguiendo esta forma de trabajar, el frontend se diseña separando las vistas completas o páginas principales de los elementos visuales y funcionales que pueden reutilizarse.

Las páginas representan los destinos definidos en el árbol de navegación, como el perfil de un jugador, la clasificación de campeones o el perfil de un profesional. Estas páginas coordinan la información y la distribuyen entre componentes más pequeños encargados de representar cada sección.

Los componentes se organizan según la información que representan, con elementos específicos para campeones, partidas y profesionales. Un ejemplo de ellos es el buscador de jugadores, que aparece tanto en la página inicial como en el perfil. La representación de estadísticas también utiliza la misma cuadrícula para jugadores y campeones. Del mismo modo, las participaciones, los objetos, las insignias de tier y los logotipos de los equipos se representan mediante componentes independientes para mantener un formato común en todas las páginas donde aparecen.

Los datos se transmiten desde las páginas hacia los componentes mediante propiedades. Cada componente recibe únicamente la información necesaria para representar su contenido y puede comunicar las acciones del usuario a la página correspondiente. De esta forma, por ejemplo, el selector de roles actualiza tanto la build recomendada como las builds profesionales.

La composición de componentes reduce la repetición, mantiene una apariencia coherente y facilita el mantenimiento de la interfaz. Cuando se modifica un elemento compartido, el cambio se refleja en todas las páginas que lo utilizan sin tener que actualizar cada una por separado. Las peticiones asíncronas y la gestión de las respuestas se explican posteriormente en el capítulo 6.

5.5.3. Adaptabilidad y accesibilidad

La interfaz se diseña para adaptarse a diferentes tamaños de pantalla mediante las utilidades responsive de Tailwind. Estas permiten modificar la distribución, el tamaño y la separación de los elementos según el espacio

disponible, sin necesidad de crear una versión independiente de cada página.

En pantallas amplias, la información puede organizarse en varias columnas para aprovechar mejor el espacio. En dispositivos con menor resolución, las columnas se colocan verticalmente, las cuadrículas reducen su número de elementos por fila y los controles ajustan su tamaño. De esta forma, las tarjetas de estadísticas, partidas, campeones y profesionales mantienen su legibilidad y pueden utilizarse también desde dispositivos móviles.

Este diseño adaptable permite disponer de una interfaz móvil funcional a partir de los mismos componentes empleados en la versión de escritorio. La navegación y las funciones disponibles se mantienen, aunque la información cambia su distribución para adaptarse al tamaño de la pantalla.

También se tienen en cuenta aspectos básicos de accesibilidad. Las imágenes incluyen textos alternativos, los controles utilizan etiquetas comprensibles y los elementos interactivos indican su estado cuando están seleccionados o deshabilitados. Además, se muestran mensajes específicos durante la carga de información y cuando se produce un error o no hay datos disponibles.

Capítulo 6

Desarrollo incremental

En este capítulo se describe el desarrollo de EasyRift siguiendo los tres incrementos definidos anteriormente. Primero se explica la preparación del proyecto y la implementación de la persistencia de datos. Después se explican las funciones añadidas en cada incremento y, finalmente, el proceso seguido para desplegar la aplicación en producción.

Nota: Este apartado es un poco raro de explicar. Me gusta el 6.2 pero a partir de ahí no me convence mucho, aunque entiendo que poner código no es viable. No sé tú como lo ves.

6.1. Preparación y organización del proyecto

Una vez definido el diseño del sistema, el desarrollo se realiza mediante tres incrementos sucesivos. Cada uno parte de la versión funcional anterior y añade nuevas entidades, servicios, endpoints y componentes a la interfaz. Antes de comenzar el primer incremento, se prepara una estructura común que permite desarrollar y ejecutar de forma independiente el backend, el frontend y la base de datos.

Como muestra la Figura 6.1, el repositorio se divide en tres directorios principales. **backend** contiene el proyecto Java realizado con Spring Boot y Maven; **frontend** contiene la aplicación React desarrollada con TypeScript y Vite; y **deploy** agrupa los archivos utilizados para ejecutar el entorno local mediante Docker Compose.

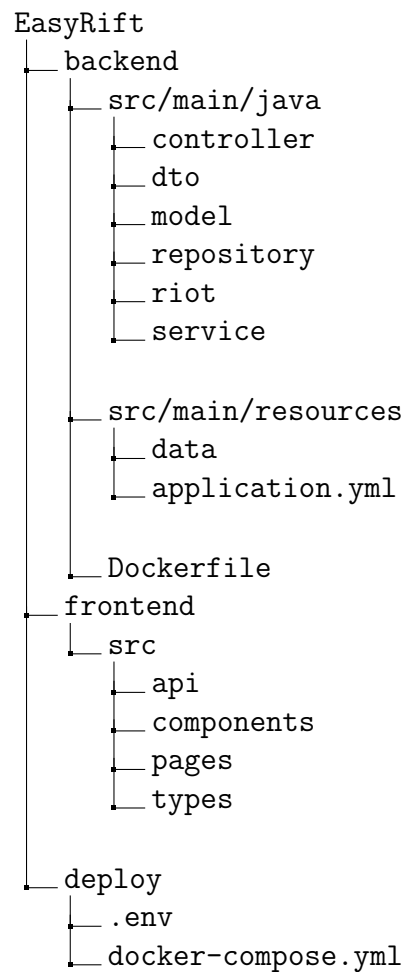


Figura 6.1: Organización simplificada del proyecto

Dentro del backend se sigue la estructura habitual de una aplicación Spring Boot. El código se distribuye entre los paquetes de modelo, repositorios, servicios, controladores, DTO, comunicación con Riot, gestión de errores, cargadores, inicializadores, planificadores y configuración. Maven gestiona las dependencias declaradas en el proyecto y permite generar el archivo ejecutable utilizado para iniciar la aplicación.

El frontend organiza su código en páginas, componentes, tipos, funciones de acceso a la API y utilidades. Las páginas representan los destinos de navegación, mientras que los componentes contienen los elementos reutilizables de la interfaz. Los tipos definen las estructuras utilizadas por TypeScript y las funciones de acceso a la API realizan las peticiones al backend.

La configuración se separa del código mediante variables de entorno. Es-

tas almacenan datos como la dirección de PostgreSQL, sus credenciales, la clave de la API de Riot y los orígenes autorizados para acceder al backend. De esta forma, se puede utilizar una configuración diferente en desarrollo y producción sin modificar el código fuente ni incluir información sensible en el repositorio de GitHub.

Durante el desarrollo local, Docker Compose inicia PostgreSQL y el backend como contenedores relacionados. La base de datos utiliza un volumen para conservar la información aunque los contenedores se detengan o se vuelvan a crear. El frontend se ejecuta mediante el servidor de desarrollo de Vite y se comunica con el backend a través de su API REST.

En una aplicación web, un origen está formado por la combinación del protocolo, el dominio y el puerto [25]. Como el frontend y el backend utilizan orígenes diferentes, CORS se configura en el backend para autorizar el origen del frontend. El navegador comprueba esta configuración antes de permitir que el frontend acceda a la respuesta.

Esta organización permite ejecutar la aplicación completa en local y comprobar la comunicación entre el frontend, el backend y PostgreSQL antes de publicar los cambios. De esta forma, cada incremento puede desarrollarse y probarse localmente antes de desplegar su nueva versión en producción.

6.2. Implementación de la persistencia de datos

La persistencia se implementa mediante JPA e Hibernate siguiendo el diagrama E-R definido durante el diseño. Cada entidad se representa mediante una clase Java y sus atributos corresponden a las columnas de la tabla asociada. Esto permite trabajar desde los servicios con objetos Java, mientras que Hibernate realiza todas las operaciones necesarias sobre PostgreSQL.

6.2.1. Entidades y relaciones

La anotación `@Entity` identifica las clases que deben almacenarse en la base de datos. Cada entidad dispone de un atributo marcado con `@Id`, que actúa como clave primaria. Cuando el identificador no procede de Riot, se utiliza `@GeneratedValue` para que se genere automáticamente.

Para explicar cómo se definen las entidades y sus relaciones mediante JPA, se utiliza como ejemplo la entidad Participación. El siguiente código

muestra sus principales atributos y anotaciones. Se omiten los constructores, getters y setters para simplificar la explicación.

```
1 @Entity
2 public class Participation {
3
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     private Integer teamId;
9     private boolean win;
10    private Integer kills;
11    private Integer deaths;
12    private Integer assists;
13
14    private Instant gameStartAt;
15
16    @Enumerated(EnumType.STRING)
17    private Role teamPosition;
18
19    @OneToOne(
20        mappedBy = "participation",
21        cascade = CascadeType.ALL,
22        orphanRemoval = true
23    )
24    private Build build;
25
26    @ManyToOne(fetch = FetchType.LAZY)
27    private Player player;
28
29    @ManyToOne(fetch = FetchType.LAZY)
30    private Match match;
31
32    @ManyToOne(fetch = FetchType.LAZY)
33    private Champion champion;
34 }
```

Código 6.1: Definición de la entidad Participación

Las relaciones `@ManyToOne` indican que varias participaciones pueden pertenecer al mismo jugador, partida o campeón. Hibernate representa estas asociaciones mediante las claves ajenas correspondientes. La carga `LAZY` evita obtener las entidades relacionadas hasta que se accede a ellas, reduciendo la cantidad de información recuperada cuando no es necesaria.

La relación con `Build` es de uno a uno. El atributo `mappedBy` indica que Participación no contiene la clave ajena de esta relación, ya que el lado propietario se encuentra en `Build`. La configuración de cascada permite aplicar

sobre la build las operaciones realizadas sobre su participación, mientras que `orphanRemoval` elimina la build cuando deja de estar asociada a una participación.

El rol se representa mediante una enumeración. La anotación `@Enumerated(EnumType.STRING)` guarda los valores literales, en lugar de almacenar la posición numérica de cada valor dentro de la enumeración.

Además de las anotaciones mostradas en la entidad, el comportamiento de Hibernate se configura en el archivo `application.yml`. Cuando se inicia el backend, Hibernate compara las entidades con el esquema existente y crea o actualiza las tablas necesarias.

6.2.2. Repositorios y consultas

El acceso a las entidades se realiza mediante repositorios de Spring Data JPA. Cada repositorio hereda de la clase `JpaRepository` e indica la entidad gestionada y el tipo de su identificador. Esta herencia proporciona automáticamente las operaciones CRUD.

Spring Data JPA también permite generar consultas a partir del nombre de los métodos. El siguiente ejemplo obtiene las participaciones de un jugador y las ordena desde la más reciente:

```
1 public interface ParticipationRepository
2     extends JpaRepository<Participation, Long> {
3
4     List<Participation>
5         findByPlayerPuuidOrderByGameStartAtDesc(String puuid);
6 }
```

Código 6.2: Repositorio de participaciones

Para los cálculos estadísticos se necesitan consultas más complejas. Estas se implementan mediante *Java Persistence Query Language* (JPQL), que utiliza las entidades y sus atributos en lugar de los nombres de las tablas y columnas de PostgreSQL.

El siguiente código muestra la consulta utilizada para obtener las estadísticas generales de un jugador. La consulta cuenta sus partidas y suma sus victorias, asesinatos, muertes y asistencias. Los resultados se devuelven mediante una proyección que agrupa estos valores para que posteriormente puedan ser procesados por el servicio de estadísticas.

```
1 @Query("""
2     SELECT new es.unex.cume.tfg.backend.repository.projection.
3     PlayerStatsAggregate(
```

```

3      COUNT(p),
4      SUM(CASE WHEN p.win = true THEN 1 ELSE 0 END),
5      SUM(p.kills),
6      SUM(p.deaths),
7      SUM(p.assists)
8  )
9  FROM Participation p
10 WHERE p.player.puuid = :puuid
11      """
12 PlayerStatsAggregate aggregatePlayerStats(
13     @Param("puuid") String puuid
14 );

```

Código 6.3: Consulta agregada de estadísticas de un jugador

El resultado se almacena en un **record**, un tipo de registro de datos de Java que agrupa varios valores relacionados y no permite modificarlos después de su creación. En este caso se utiliza como proyección para recibir los resultados de la consulta. De esta forma, PostgreSQL realiza las operaciones de agrupación y devuelve únicamente los valores necesarios, sin cargar todas las participaciones para recorrerlas posteriormente en Java.

La combinación de entidades, repositorios y consultas JPQL permite implementar tanto las operaciones sencillas de persistencia como los cálculos para las estadísticas y recomendaciones.

6.3. Desarrollo del primer incremento

El primer incremento establece la comunicación con Riot Games, implementa el almacenamiento de jugadores y partidas, expone las primeras operaciones de la API REST y construye la interfaz inicial para consultar esta información.

6.3.1. Comunicación con Riot Games

La comunicación con Riot Games se separa de la lógica de negocio mediante una capa específica. Esta contiene los clientes HTTP, los DTO que representan las respuestas de la API externa, la construcción de las direcciones y el tratamiento de los errores devueltos por Riot.

La API de Riot Games divide sus operaciones en diferentes servicios y grupos de endpoints según el tipo de información, como cuentas, partidas o partidas en curso. Esta misma separación se reproduce en el backend mediante un

cliente HTTP especializado para cada grupo. Por ejemplo, `AccountClient` obtiene el PUUID asociado a un Riot ID, `MatchClient` consulta identificadores y detalles de partidas, y `SpectatorClient` comprueba si un jugador se encuentra jugando.

Todos los clientes utilizan una instancia compartida de `RestClient`. Su configuración añade la clave de la API, indica que las respuestas deben recibirse en formato JSON y establece tiempos máximos de conexión y lectura.

Riot utiliza direcciones diferentes en función del servicio consultado. Algunas peticiones se realizan sobre una plataforma concreta, como `euw1`, mientras que otras requieren una región de enrutamiento, como `europa`.

Las respuestas JSON se convierten en DTO de Riot antes de ser utilizadas por los servicios. Estos DTO contienen únicamente los campos deseados y separan la estructura de la API externa de las entidades almacenadas por EasyRift. Si la respuesta contiene un error, se genera una excepción propia que conserva el código HTTP, lo que permite tratar problemas como los recursos inexistentes o el límite de peticiones.

Finalmente, se utiliza un servicio llamado `RiotFetchService` que centraliza el acceso a estos clientes y proporciona al resto de los servicios un único punto desde el que solicitar información a Riot.

6.3.2. Consulta y sincronización de jugadores

La búsqueda de jugadores se implementa en `PlayerService` a partir de la plataforma, el nombre de juego y la etiqueta del jugador. Primero se comprueba si ya se encuentra almacenado. Cuando existe, se reutiliza su PUUID sin realizar una nueva consulta a Riot. En caso contrario, el PUUID se obtiene mediante el Riot ID proporcionado.

Una vez conocido el PUUID, se vuelve a buscar al jugador mediante este identificador. Esta segunda comprobación permite encontrar cuentas que ya se han almacenado como participantes al cargar partidas de otros jugadores o cuyo Riot ID ha cambiado. Si el jugador dispone de todos sus datos, se devuelve directamente; si no existe o su información está incompleta, se inicia su sincronización.

Durante la sincronización, `PlayerService` obtiene el nivel y el icono de perfil. También determina la fecha desde la que deben buscarse nuevas partidas: se utiliza la última sincronización cuando está disponible y, para un jugador nuevo, se establece como límite el último año. La carga de las partidas se delega en `MatchService` y se limita a un máximo de veinte por

sincronización para controlar el número de peticiones realizadas a Riot.

MatchService solicita primero los identificadores de las partidas posteriores a la fecha indicada. Para cada uno comprueba si la partida ya está almacenada y, cuando no existe, consulta su información completa. El DTO recibido se transforma en una entidad **Partida** y se guardan también sus diez participaciones mediante **ParticipationService**. El identificador proporcionado por Riot se utiliza como clave primaria, por lo que la base de datos impide almacenar dos veces la misma partida.

Cada participación se relaciona con la partida, el jugador, el campeón y la build correspondientes. Los jugadores que aparecen entre los participantes también se almacenan con sus datos básicos cuando todavía no existen. De esta forma, pueden mostrarse y utilizarse como enlaces desde los detalles de una partida, aunque su perfil completo no se sincroniza hasta que un usuario lo consulta directamente.

La consulta de una partida en curso se implementa de forma independiente mediante **CurrentGameService**. Este servicio utiliza el cliente de espectador de Riot y devuelve la información disponible cuando el jugador está jugando. Si Riot responde con un código 404, se interpreta que no existe una partida activa y se comunica este estado al frontend.

Finalmente, las operaciones que modifican la información de un jugador se protegen mediante un cerrojo asociado a su PUUID. Antes de comenzar el procesamiento, se obtiene el cerrojo correspondiente y, al finalizar, se libera. Esto garantiza la exclusión mutua y evita que dos peticiones sincronicen simultáneamente al mismo jugador.

6.3.3. API y transferencia de datos

La API REST se implementa mediante controladores. Cada controlador contiene las operaciones relacionadas con un elemento de la aplicación. Durante el primer incremento se crean los controladores de jugadores, partidas y campeones, que posteriormente se amplían con las funcionalidades de los siguientes incrementos.

Las clases se identifican mediante la anotación **@RestController** y utilizan **@RequestMapping** para establecer la ruta de sus endpoints. Las operaciones HTTP se definen con anotaciones como **@GetMapping** y **@PostMapping**. Los valores enviados por el frontend se reciben mediante parámetros o variables incluidas en la ruta y Spring los convierte automáticamente al tipo requerido.

Los controladores se separan de la lógica de negocio. Su función consiste en recibir los datos de la petición, solicitar la operación al servicio correspondiente y construir la respuesta HTTP. Por ejemplo, `PlayerController` utiliza los servicios de jugadores y participaciones para obtener la información necesaria para el perfil, pero no realiza la sincronización ni accede a los repositorios.

Las entidades almacenadas no se devuelven directamente al frontend. En su lugar, se utilizan DTO implementados mediante estructuras `record`, cuyos valores se establecen al crearlos y no se modifican posteriormente. Cada DTO contiene únicamente los datos necesarios para una respuesta determinada y métodos encargados de transformar las entidades al formato del DTO en cuestión. Esta separación evita exponer la estructura interna de la base de datos y permite modificar las entidades sin cambiar las respuestas de la API.

La gestión de errores se centraliza mediante `GlobalExceptionHandler`. Cuando un servicio genera una excepción, este componente selecciona el código HTTP correspondiente y construye un `ApiErrorDto`. De esta forma, los controladores no necesitan repetir el tratamiento de errores y el frontend puede procesar las respuestas incorrectas.

6.3.4. Primera interfaz web

La interfaz del primer incremento se desarrolla mediante React y se divide en las páginas correspondientes al buscador, al perfil del jugador, al listado de campeones y al detalle de un campeón. Estas páginas se asocian a sus rutas mediante React Router y reciben los datos necesarios para identificar al jugador o campeón consultado a través de la URL.

La comunicación con el backend se realiza en `backendApi.ts`. Este archivo contiene una función común que construye la dirección del backend, realiza la petición mediante `fetch` y convierte la respuesta JSON al tipo de TypeScript correspondiente. Sobre esta función se definen las operaciones concretas para buscar jugadores, actualizar perfiles, consultar partidas y obtener campeones.

Los tipos utilizados en el frontend reproducen exactamente la estructura de los DTO devueltos por el backend. Cuando el backend devuelve una respuesta incorrecta, se genera un `ApiError` que conserva el código, el estado HTTP y el mensaje recibido.

Las peticiones al backend son asíncronas y devuelven promesas. Una promesa es un objeto que representa el resultado futuro de una operación que

todavía no ha finalizado. Inicialmente se encuentra pendiente y, cuando termina, puede resolverse con el valor obtenido o rechazarse debido a un error. Esto permite realizar peticiones HTTP sin bloquear el funcionamiento del navegador mientras se espera la respuesta.

Las páginas utilizan `async` y `await` para trabajar con las promesas. `await` detiene temporalmente la ejecución de la función hasta recibir el resultado, pero no bloquea el resto de la interfaz. Mientras se procesa la petición se muestra un estado de carga y, si se produce un error, se presenta un mensaje en lugar del contenido esperado. Cuando la promesa se resuelve, los datos se almacenan en el estado de la página y React actualiza automáticamente los componentes afectados.

El buscador está formado por la plataforma, el nombre y la etiqueta, y utiliza React Router para navegar a la ruta del jugador. La página del perfil obtiene estos valores de la URL y solicita al backend sus datos. El resultado se distribuye entre componentes encargados de mostrar la información básica, el estado de la partida actual y el historial de partidas. Los detalles de cada partida se solicitan cuando el usuario despliega la participación correspondiente.

El listado de campeones se obtiene también desde el backend y permite acceder a la página asociada a cada identificador. Las imágenes de campeones, objetos, hechizos e iconos de perfil se cargan desde Data Dragon utilizando los identificadores incluidos en los DTO. Para evitar utilizar una versión desactualizada de estos recursos, el frontend consulta primero la última versión disponible y construye a partir de ella las direcciones correspondientes.

Para evitar diferenciar los objetos únicamente por su imagen, se implementa un panel que muestra sus características al situar el cursor sobre ellos o al seleccionarlos en una pantalla táctil. La información se obtiene previamente de Data Dragon y se almacena en memoria, evitando realizar una nueva petición cada vez que se muestra el panel. Esto facilita la identificación de los objetos y permite conocer sus características en detalle.

6.4. Desarrollo del segundo incremento

El segundo incremento amplía la información disponible a partir de las partidas almacenadas. Se añaden los rangos clasificatorios y las estadísticas de los jugadores; se calculan estadísticas globales de los campeones y se automatiza la actualización periódica de los datos. Finalmente, el frontend se adapta para mostrar esta nueva información en los perfiles de jugadores y

campeones.

6.4.1. Rangos y estadísticas de jugadores

El perfil del jugador se amplía con sus rangos clasificatorios y con estadísticas calculadas a partir de las participaciones almacenadas en la base de datos. Para representar esta información se añaden las entidades **RankedRank** y **PlayerStats**, ambas relacionadas con el jugador.

Los rangos se obtienen a través de los endpoints de League de Riot mediante **LeagueClient**. Riot devuelve la cola clasificatoria, el tier, la división, los puntos de liga, las victorias y las derrotas. Estos datos se transforman en entidades **RankedRank**, permitiendo separar la clasificación de Solo/Duo y la de Flex.

Cuando se actualiza el jugador, se eliminan los rangos guardados y se almacenan los valores actuales proporcionados por Riot. La aplicación conserva únicamente el estado clasificatorio actual del jugador. Este proceso se añade a la sincronización descrita en el primer incremento.

Las estadísticas se calculan mediante **PlayerStatsService** a partir de las participaciones del jugador. El uso de consultas agregadas permite obtener el número de partidas, las victorias y los totales de asesinatos, muertes y asistencias sin cargar todas las participaciones en memoria. A partir de estos valores se calculan las derrotas, el porcentaje de victorias, el KDA y los promedios por partida.

También se determina el mejor campeón del jugador. Para ello, sus participaciones se agrupan por campeón y se comparan mediante la puntuación definida en el diseño, que combina la tasa de victorias y la tasa de selección normalizada. De esta forma, la elección tiene en cuenta tanto los resultados obtenidos como la frecuencia de uso de cada campeón.

Los resultados se almacenan en una entidad **PlayerStats** asociada al jugador mediante su PUUID. Si ya existen estadísticas, se actualizan; si todavía no existen, se crean. Finalmente, **PlayerDetailsDto** incluye tanto los rangos como las estadísticas para enviarlos junto con el resto de la información del perfil.

6.4.2. Estadísticas de campeones

También se añaden estadísticas para los campeones a partir de las participaciones almacenadas. Para guardar estos resultados, se crea la entidad

ChampionStats, relacionada con **Champion**, que utiliza el identificador del campeón como clave primaria.

El cálculo se realiza mediante el servicio **ChampionStatsService**. Una consulta agrupa las participaciones por campeón y obtiene el número de partidas, las victorias y los totales de asesinatos, muertes y asistencias. A partir de estos datos se calculan las derrotas, la tasa de victorias, el KDA y los promedios por partida.

Cuando un campeón ya tiene estadísticas almacenadas, se actualizan sus valores. Si todavía no existen, se crea una nueva entidad **ChampionStats**. A diferencia de las estadísticas de los jugadores, este cálculo se realiza para todos los campeones, ya que depende del total de partidas disponible en la base de datos.

Finalmente, las estadísticas se añaden al DTO utilizado para consultar los detalles de un campeón. De esta forma, el frontend puede mostrarlas junto con su información básica y sus builds recientes.

6.4.3. Automatización

La actualización de los profesionales y el cálculo de las estadísticas de campeones se realizan automáticamente mediante los planificadores de Spring. Esta funcionalidad se activa con la anotación **@EnableScheduling** y permite ejecutar operaciones de forma periódica sin que el usuario tenga que solicitarlas.

ProfessionalScheduler se encarga de actualizar los datos de los jugadores profesionales. Estos se ordenan por la fecha de su última sincronización para comenzar por quienes llevan más tiempo sin actualizarse. A continuación, se utiliza el mismo proceso que en los perfiles normales para cargar sus nuevas partidas, actualizar sus rangos y recalculan sus estadísticas. El orden utilizado evita que siempre queden pendientes los profesionales situados al final de la lista.

ChampionStatsScheduler vuelve a calcular periódicamente las estadísticas de todos los campeones utilizando las partidas almacenadas. Así, los resultados se actualizan después de añadir nuevas partidas y las consultas del frontend solo tienen que leer los valores ya calculados.

Los intervalos se configuran en **application.yml** y pueden modificarse mediante variables de entorno. Las estadísticas de los jugadores no se recalculan de forma global, sino únicamente cuando se sincroniza cada perfil. De esta forma, se evita recorrer una cantidad de jugadores que puede crecer

notablemente con el tiempo.

6.4.4. Ampliación del frontend

El frontend se adapta para mostrar los rangos y las estadísticas añadidos al perfil del jugador. El tipo `PlayerDetailsDto` se amplía con los nuevos datos, por lo que la página puede recibirlos junto con la información básica y el historial mediante la misma petición.

La nueva información se organiza en una barra lateral. Primero se muestra el estado de la partida actual, seguido de las clasificaciones de Solo/Duo y Flex y, finalmente, las estadísticas del jugador. Cada rango incluye el tier, la división, los puntos de liga, el porcentaje de victorias y el número de victorias y derrotas. Si el jugador no tiene clasificación en una de las colas, la tarjeta correspondiente no se muestra.

La tarjeta de estadísticas muestra primero el mejor campeón del jugador y, a continuación, sus resultados generales. Estos valores se distribuyen mediante una cuadrícula reutilizable para mantener el mismo estilo en toda la aplicación.

La página de detalle del campeón también se amplía con sus estadísticas globales. Para mostrarlas, se reutiliza la misma cuadrícula del perfil del jugador, adaptando únicamente los datos recibidos. Si todavía no se han calculado estadísticas, se muestra un mensaje en su lugar.

6.5. Desarrollo del tercer incremento

En el tercer incremento, se mejora la gestión de los jugadores profesionales y se añaden la clasificación de campeones por tiers y las builds recomendadas por campeón y rol. El frontend también se amplía con las páginas y controles necesarios para consultar estas nuevas funciones.

6.5.1. Inicialización de profesionales

En los incrementos anteriores, la plantilla de jugadores profesionales se encuentra directamente en el código fuente. En el tercer incremento se traslada al archivo `professionals.json`, lo que permite modificar los jugadores, equipos, ligas y roles sin cambiar las clases del backend.

Al iniciar la aplicación, la plantilla del archivo se compara con los profesionales almacenados en la base de datos. Si un profesional ya existe, se actualizan sus datos. Si es nuevo, se utiliza su Riot ID para obtener el PUUID y crear la relación entre las entidades **Professional** y **Player**.

En esta fase solo se obtienen los datos básicos necesarios para registrar al jugador, como su nombre, plataforma, nivel e icono de perfil. No se cargan todavía sus partidas, rangos ni estadísticas, por lo que la fecha de sincronización de los nuevos jugadores queda vacía. Esto reduce el número de peticiones necesarias durante el arranque.

Después de la inicialización, **ProfessionalScheduler** encuentra primero a estos jugadores sin sincronizar y ejecuta sobre ellos el proceso descrito anteriormente. De esta forma, la creación de la plantilla se realiza una vez al iniciar la aplicación, mientras que la actualización de sus partidas se reparte entre las ejecuciones periódicas del planificador.

Cuando un profesional deja de aparecer en **professionals.json**, se elimina únicamente su asociación como profesional. Su jugador, sus partidas y sus participaciones se conservan porque siguen siendo útiles para los cálculos generales.

6.5.2. Clasificación de campeones

En este incremento se añade una clasificación por tiers para ordenar los campeones según los datos almacenados. Para representarla, se crea la enumeración **Tier**, formada por los valores S, A, B, C, D y E, y se añade a la entidad de estadísticas de campeón.

Antes de realizar los cálculos, se obtiene el parche más reciente entre las partidas clasificatorias almacenadas. A continuación, solo se utilizan las participaciones de ese parche pertenecientes a las colas Solo/Duo y Flex. De esta forma, no se mezclan partidas normales ni datos de versiones anteriores del juego.

Las estadísticas de los campeones se vuelven a calcular con este conjunto de partidas. Para cada campeón se obtiene una puntuación mediante la fórmula definida en el diseño, que combina la tasa de victorias y la tasa de selección normalizada. La tasa de selección se normaliza dividiendo sus partidas entre las del campeón más utilizado.

Una vez calculadas las puntuaciones, los campeones se ordenan de mayor a menor. El 10 % con mayor puntuación recibe el tier S; los siguientes se distribuyen en grupos del 20 % entre los tiers A, B, C y D; y el 10 % final

recibe el tier E. Los campeones que no tienen partidas en el parche actual se asignan al tier C como valor neutral.

El tier calculado se guarda junto con las estadísticas y se añade a los DTO de campeón. De esta forma, el listado puede mostrar directamente la clasificación guardada sin repetir los cálculos cada vez que se realiza una consulta. Este proceso se incorpora a la actualización periódica ejecutada por `ChampionStatsScheduler`.

6.5.3. Generación de builds recomendadas

Para guardar las recomendaciones se añade la entidad `RecommendedBuild`. Aunque contiene información similar a `Build`, su función es diferente: una build representa los objetos utilizados en una participación concreta, mientras que una recomendación representa un conjunto de objetos asociados con un campeón y un rol pero no con una participación.

El cálculo utiliza únicamente las partidas clasificatorias del parche más reciente. Se exige un mínimo de diez partidas para generar una recomendación para una combinación de campeón y rol. De esta forma, se evita hacer una recomendación a partir de una cantidad demasiado pequeña de datos.

Los seis objetos principales se ordenan antes de guardar cada build. Así, dos combinaciones formadas por los mismos objetos se consideran iguales aunque el jugador los coloque en posiciones diferentes del inventario. Posteriormente, una consulta agrupa las builds coincidentes y obtiene el número total de partidas y victorias de cada una.

También se comprueba que los seis objetos sean objetos completos. Para ello, se carga el catálogo local `item.json` y se descartan los componentes intermedios y las builds con menos de seis objetos.

Las builds válidas se comparan mediante la puntuación definida en el diseño. La tasa de victorias se calcula a partir de sus victorias y la tasa de selección se normaliza respecto a la build más utilizada para el mismo campeón y rol. La configuración con mayor puntuación se selecciona como recomendación y, en caso de empate, se elige la que aparece en más partidas.

El resultado se guarda junto con el campeón, el rol, los objetos, los hechizos de invocador y el número total de partidas del campeón en esa posición. Este último valor permite ordenar las recomendaciones y utilizar por defecto el rol más jugado. Las recomendaciones anteriores que ya no se pueden calcular con los datos del parche actual se eliminan.

Finalmente, `ChampionStatsScheduler` se amplía para generar las builds

recomendadas después de actualizar las estadísticas y los tiers. De esta forma, las recomendaciones se recalculan periódicamente a medida que se añaden nuevas partidas a la base de datos.

6.5.4. Interfaz final

El listado de campeones se adapta para mostrar la clasificación calculada en el backend. Los campeones se separan en secciones para los diferentes tiers. El buscador permite filtrar los campeones por nombre sin eliminar esta organización.

La página de cada campeón cuenta con un selector con los cinco roles. Los roles que tienen una recomendación disponible pueden seleccionarse, mientras que los demás aparecen desactivados. Al entrar en la página se selecciona automáticamente el rol con más partidas y se muestran su build recomendada y las builds recientes de profesionales para esa posición. Cuando el usuario cambia de rol, ambos apartados se actualizan con los datos que ya se han recibido del backend.

También se añade un apartado para consultar jugadores profesionales. En la primera página aparecen agrupados por equipos y ordenados según su posición. Cada equipo muestra su nombre, su logotipo y sus jugadores. Al seleccionar uno de ellos se accede a un perfil profesional independiente del perfil normal de jugador.

El perfil profesional muestra el nombre competitivo, el Riot ID, el equipo, la liga y el rol del jugador, además de sus veinte builds clasificatorias más recientes. Desde esta página también se puede acceder al perfil normal de su cuenta para consultar los rangos, las estadísticas y el historial de partidas. Los nombres que aparecen en las builds recientes de un campeón también enlazan con el perfil profesional correspondiente.

6.6. Despliegue a producción

Una vez comprobado el funcionamiento de la aplicación en local, se publica cada componente siguiendo la arquitectura definida durante el diseño. Para proporcionar una dirección propia en la aplicación, se compra el dominio EasyRift.es mediante IONOS y se configura para dirigir a los usuarios hacia el frontend.

El frontend se despliega en Vercel utilizando su plan gratuito. El proyecto

se conecta con el repositorio de GitHub y Vercel se encarga de construir y publicar los archivos resultantes. El dominio se vincula al proyecto desde el panel de la plataforma, que también genera automáticamente el certificado HTTPS [26].

El backend se despliega como un servicio web en Render. La plataforma utiliza el Dockerfile incluido en el repositorio para construir una imagen y ejecutar el contenedor resultante. PostgreSQL se crea como un servicio independiente dentro de la misma plataforma. El backend se conecta a la base de datos mediante la dirección interna que ofrece Render [23].

Las credenciales de PostgreSQL, la clave de la API de Riot y los orígenes autorizados se configuran mediante variables de entorno. De esta forma, la configuración de producción se mantiene separada del código fuente y la información sensible no se incluye en el repositorio.

Tanto Render como Vercel se conectan directamente con GitHub. Cuando se envían cambios a la rama principal, cada plataforma vuelve a construir y desplegar automáticamente el componente correspondiente. Esto permite mantener la aplicación publicada sincronizada con el repositorio sin ejecutar manualmente el proceso de publicación.

El backend y PostgreSQL utilizan servicios de pago en Render. Su coste depende de los recursos contratados y del tiempo que permanecen activos. Inicialmente, se seleccionan recursos ajustados al volumen de uso previsto durante las pruebas.

Capítulo 7

Evaluación de la aplicación final

Una vez completados los tres incrementos y desplegada la versión final de EasyRift, se realizan distintas pruebas para comprobar su funcionamiento y verificar el cumplimiento de los objetivos y requisitos definidos. La evaluación se lleva a cabo sobre la aplicación publicada, por lo que también permite comprobar la comunicación entre el frontend, el backend, PostgreSQL y los servicios externos de Riot Games.

7.1. Entorno y método de evaluación

La evaluación se realiza sobre la versión de EasyRift accesible mediante el dominio <https://easyrift.es>. Esta versión utiliza el frontend desplegado en Vercel y el backend y PostgreSQL alojados en Render.

El código fuente del proyecto se encuentra disponible en GitHub, desde donde Render y Vercel realizan los despliegues automáticos. El repositorio es público y accesible en https://github.com/fmonteroi/TFG_Informatica.

Las pruebas funcionales se basan en los recorridos principales que puede realizar un usuario: buscar un jugador, consultar su perfil y sus partidas, ver la clasificación de campeones junto con los detalles de cada campeón y acceder a la información de los jugadores profesionales. También se comprueba el funcionamiento de los procesos automáticos que actualizan los profesionales y calculan las estadísticas, los tiers y las builds recomendadas.

En cada prueba se realizan una serie de acciones concretas y se compara el comportamiento observado con el resultado esperado. Además, se incluyen capturas de la aplicación como evidencia cuando es necesario. Los procesos internos que no se muestran directamente en el frontend se comprueban

mediante la consola del backend y los datos guardados en PostgreSQL.

La interfaz también se comprueba con diferentes tamaños de pantalla para confirmar que sus elementos se reorganizan correctamente y que las funciones principales siguen siendo accesibles. Finalmente, los resultados se comparan con los requisitos definidos anteriormente para determinar cuáles se cumplen e identificar las limitaciones del sistema.

7.2. Validación funcional mediante casos de uso

Para la validación funcional se realizan los principales recorridos de navegación de EasyRift. Para cada uno se ejecutan las acciones disponibles y se comprueba que el resultado obtenido coincide con el comportamiento esperado. Además, las capturas permiten mostrar todas las páginas que forman la versión final de la aplicación.

7.2.1. Consulta de perfiles de jugadores

El primer recorrido comienza en la página principal, mostrada en la Figura 7.1. Al acceder aparece el aviso que informa de que EasyRift se encuentra en fase de pruebas y pendiente de aprobación por Riot Games. Después de cerrarlo, el usuario puede introducir la plataforma y el Riot ID.

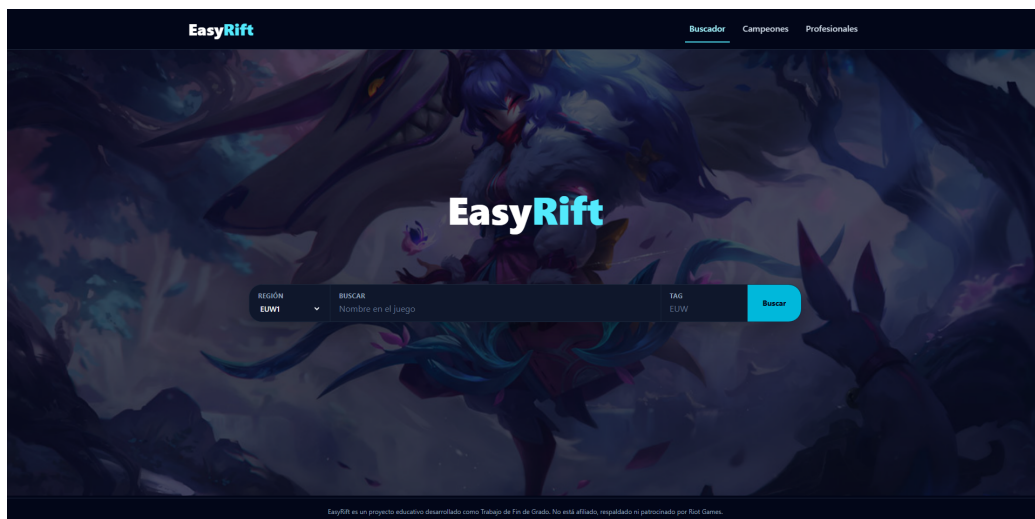


Figura 7.1: Buscador de jugadores

Para validar la búsqueda se introduce un Riot ID existente y se comprueba que la aplicación muestra el perfil correspondiente. Como el jugador se consulta por primera vez, el backend obtiene su información y la almacena antes de devolver los datos necesarios. El resultado de la prueba es correcto, ya que el perfil aparece sin que el usuario tenga que realizar ninguna operación adicional.

La Figura 7.2 muestra el perfil obtenido. En la parte superior aparecen el Riot ID, la plataforma, el nivel y el icono del jugador. La barra lateral muestra el estado de la partida actual, los rangos clasificatorios disponibles y las estadísticas calculadas. El resto de la página está formado por el historial de partidas recientes y la opción para refrescar el perfil del jugador.

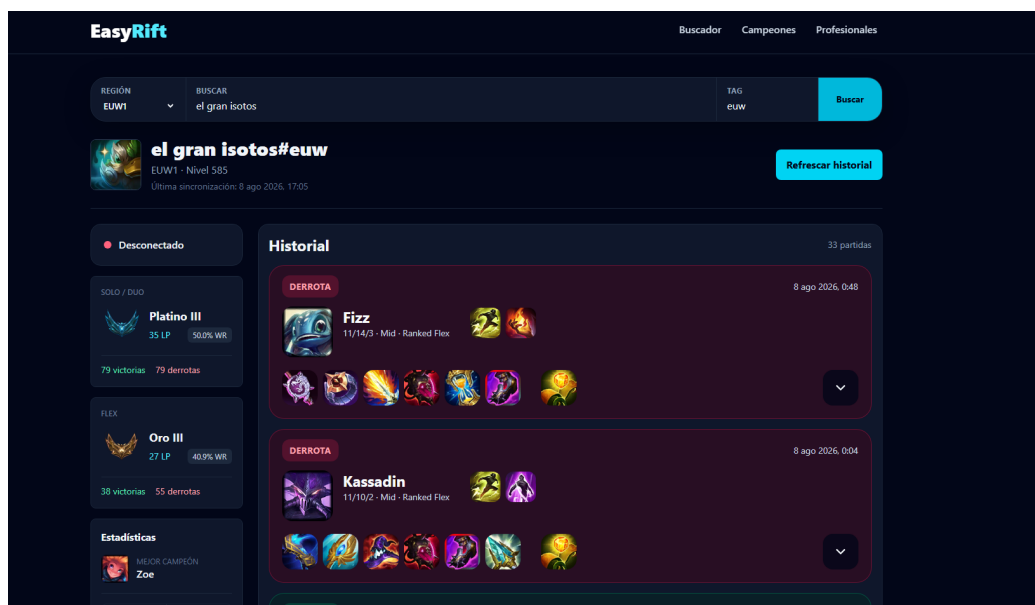


Figura 7.2: Perfil de jugador

La búsqueda se repite seleccionando la plataforma coreana e introduciendo el Riot ID del jugador profesional Faker. Como se observa en la Figura 7.3, la aplicación también localiza y muestra correctamente jugadores pertenecientes a una plataforma diferente de la europea. Esta prueba permite comprobar que la plataforma seleccionada se tiene en cuenta al realizar la consulta.

EasyRift

Buscador

Campeones

Profesionales

REGIÓN

KR

BUSCAR

차가운코끼리

TAG

KRO

Buscar

차가운코끼리#KRO

KR · Nivel 1218

Ultima sincronización: 8 ago 2026, 17:13

Refrescar historial

Desconectado

SOLO / DUO

Challenger

2070 LP

50.9% WR

799 victorias

770 derrotas

Estadísticas

MEJOR CAMPEÓN

Jayce

PARTIDAS	WIN RATE
20	55.0%
VICTORIAS	DERROTAS
11	9
KDA	KILLS
2.33	5.8

Historial

20 partidas

VICTORIA

7 ago 2026, 20:53

Jayce

11/4/6 · Top · Ranked Solo/Duo

DERROTA

7 ago 2026, 20:12

Garen

2/8/4 · Top · Ranked Solo/Duo

Figura 7.3: Consulta de un jugador de la plataforma coreana

Para comprobar la consulta de una partida en curso se accede al perfil de un jugador que se encuentra jugando en ese momento. La tarjeta mostrada en la Figura 7.4 indica que el jugador está en partida y muestra el campeón utilizado, el tipo de cola y el tiempo transcurrido. Cuando el jugador no se encuentra jugando, la misma tarjeta muestra el estado desconectado.

74

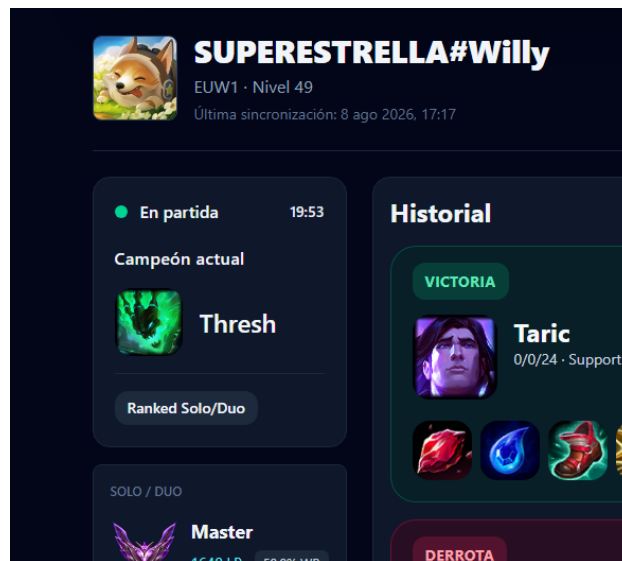


Figura 7.4: Consulta de una partida en curso

En el historial se comprueba que cada participación muestra el resultado, el campeón, el KDA, el rol, la cola, los hechizos y los objetos utilizados. Si el rol o alguno de los rangos no están disponibles, el elemento correspondiente no se muestra y la estructura de la página se mantiene correctamente.

Al desplegar una participación se solicitan los detalles completos de la partida. Como se observa en la Figura 7.5, se muestran los dos equipos y sus diez participantes. Los nombres funcionan como enlaces, por lo que también se comprueba que al seleccionar otro participante se navega hasta su perfil.

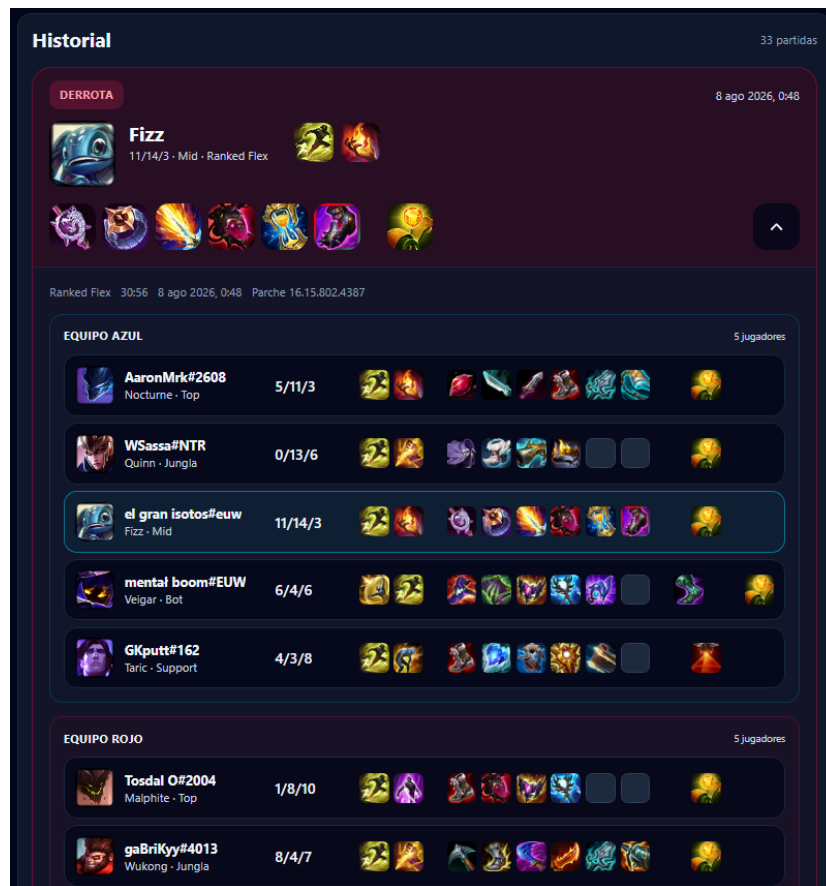


Figura 7.5: Detalles de partida

Finalmente, se utiliza la opción de refrescar el perfil. La aplicación vuelve a sincronizar al jugador y modifica la fecha de la última sincronización sin duplicar las partidas que ya estaban almacenadas.

7.2.2. Consulta de campeones

El segundo recorrido comienza en el listado de campeones mostrado en la Figura 7.6. Los campeones aparecen agrupados según los tiers S, A, B, C, D y E calculados por el backend. Cada uno se representa mediante su nombre e imagen y funciona como enlace hacia su página de detalle.

La barra de búsqueda permite filtrar los campeones por nombre. Para validar su funcionamiento, se introduce parte del nombre de un campeón y se comprueba que solo aparecen las coincidencias, manteniendo visibles las secciones de todos los tiers.

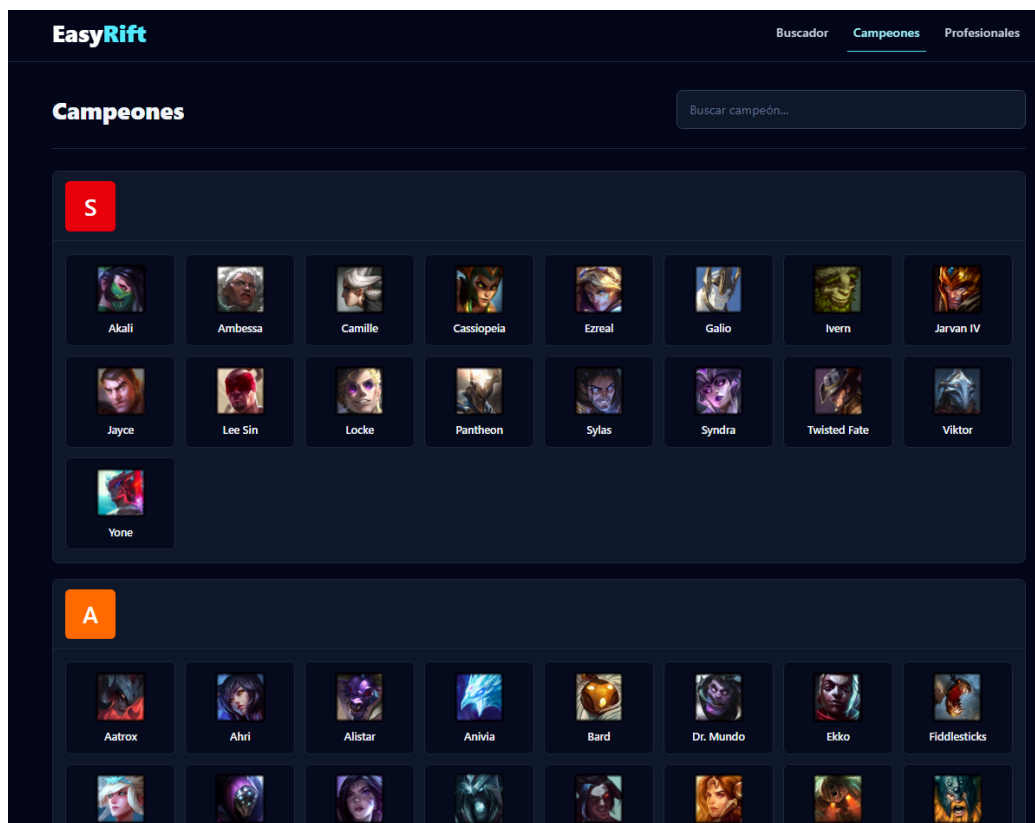


Figura 7.6: Clasificación de campeones por tiers

Al seleccionar un campeón se accede a la página de detalle mostrada en la Figura 7.7. En la cabecera aparecen su nombre, imagen y tier, mientras que el resto de la página presenta sus estadísticas, los roles disponibles, la build recomendada y las builds recientes de profesionales.

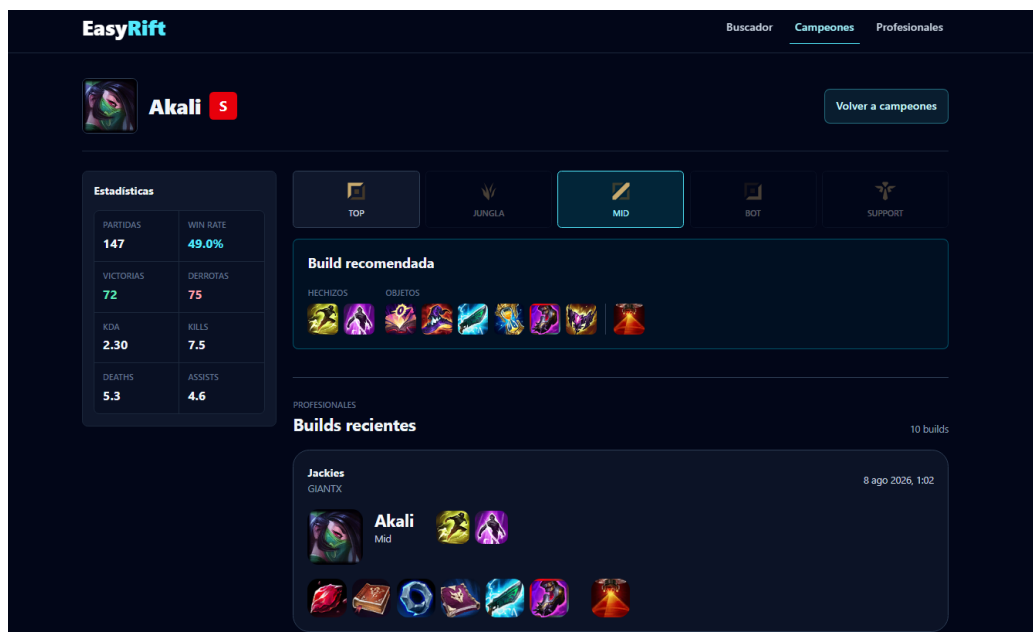


Figura 7.7: Detalle de un campeón y selección de rol

Los roles que cuentan con suficientes partidas pueden seleccionarse, mientras que los demás aparecen deshabilitados. Al acceder a la página se selecciona automáticamente el rol que cuenta con más partidas. Para comprobar este comportamiento, se cambia el rol seleccionado. Como se observa en la Figura 7.8, tanto la recomendación como las builds profesionales se actualizan con la información correspondiente.

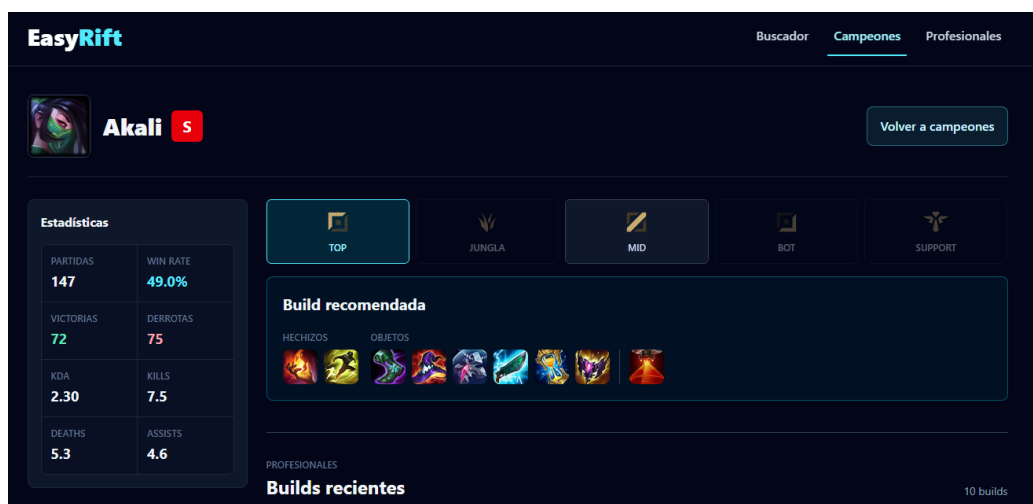


Figura 7.8: Detalle de un campeón en otro rol

La build recomendada muestra los hechizos de invocador y los objetos seleccionados para el campeón y el rol. Las builds profesionales incluyen además el nombre del jugador, su equipo y la fecha de la partida. Al seleccionar el nombre de un profesional se navega correctamente hasta su perfil profesional.

Finalmente, se sitúa el cursor sobre uno de los objetos para comprobar que aparece un panel con su nombre y descripción. Como muestra la Figura 7.9, esta información se presenta sin abandonar la página del campeón y desaparece al retirar el cursor.



Figura 7.9: Información detallada de un objeto

Las pruebas realizadas muestran que el listado, el buscador, la navegación hacia cada campeón, la selección por rol y la consulta de builds funcionan de acuerdo con el comportamiento esperado.

7.2.3. Consulta de jugadores profesionales

El tercer recorrido comienza con el listado de profesionales mostrado en la Figura 7.10. Los jugadores aparecen agrupados por equipos y ordenados según su rol. Cada tarjeta muestra el nombre, el logo y la liga del equipo, junto con los profesionales que forman parte de su plantilla.

Se comprueba que los equipos y jugadores definidos en el backend aparecen en la interfaz de la aplicación. Cuando no se conoce la cuenta de uno de los profesionales, se muestra como desconocido y no se permite acceder a su perfil. El resto de los nombres funcionan como enlaces hacia el perfil profesional correspondiente.

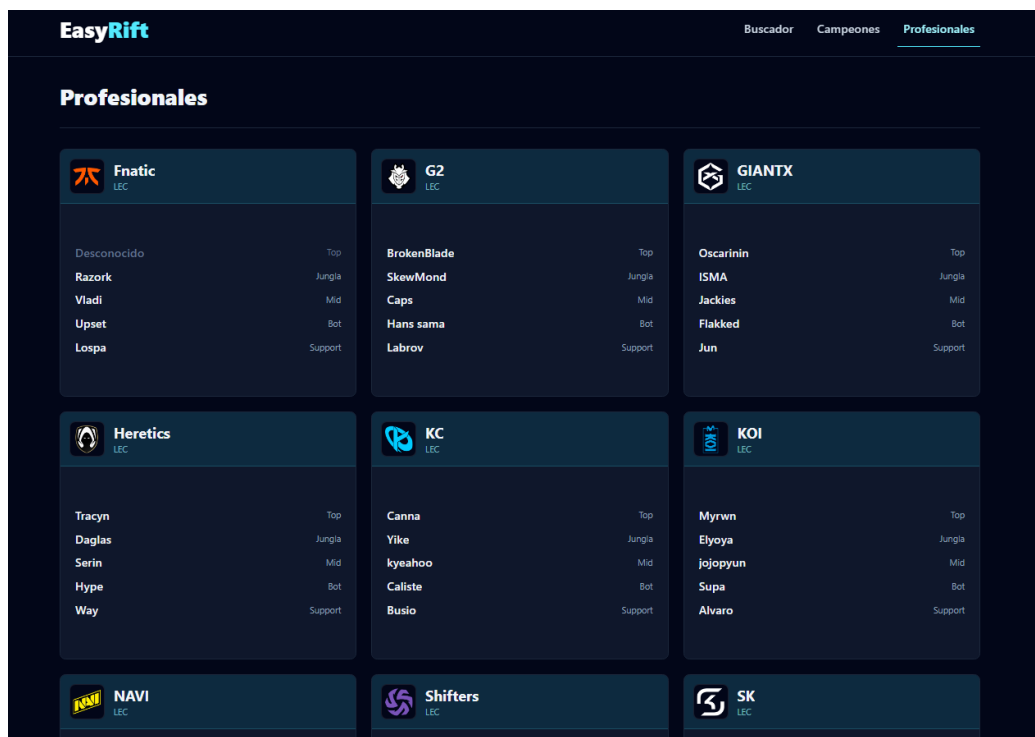


Figura 7.10: Jugadores profesionales agrupados por equipos

Para validar la navegación se selecciona al jugador Elyoya. Como se observa en la Figura 7.11, la aplicación muestra un perfil diferenciado del perfil normal de jugador. En la cabecera aparecen su nombre profesional y su Riot ID, mientras que las tarjetas inferiores indican su equipo, rol y liga.

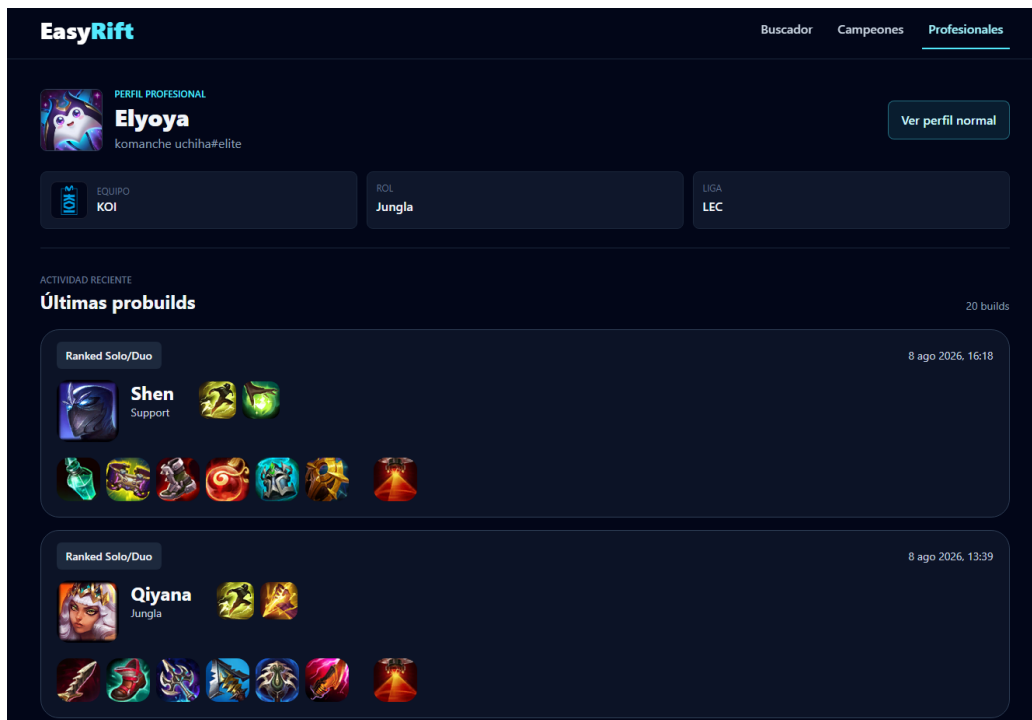


Figura 7.11: Perfil de un jugador profesional

La parte inferior de la página muestra sus veinte builds clasificatorias más recientes. De esta forma, se comprueba que la información almacenada durante las actualizaciones automáticas de los profesionales puede consultarse correctamente desde el frontend.

Finalmente, se utiliza la opción para acceder al perfil normal del jugador. La aplicación navega hasta la vista de perfil evaluada anteriormente, donde se pueden consultar sus rangos, estadísticas, estado actual e historial de partidas.

7.2.4. Procesos automáticos y base de datos

Los procesos automáticos que no dependen de una acción del usuario se comprueban mediante la consola del backend desplegado en Render. Como muestra la Figura 7.12, el planificador inicia periódicamente la actualización de los profesionales y muestra el número de jugadores actualizados al finalizar. En la prueba se completan correctamente las 48 actualizaciones previstas.

```

10:20:40 AM [h8lp2] Professional refresh completed: 48/48
10:40:40 AM [h8lp2] Starting professional refresh
10:40:48 AM [h8lp2] Professional refresh completed: 48/48
10:55:24 AM [h8lp2] Starting champion data calculation
10:55:25 AM [h8lp2] Champion data calculation completed
11:00:48 AM [h8lp2] Starting professional refresh
11:00:58 AM [h8lp2] Professional refresh completed: 48/48
11:20:58 AM [h8lp2] Starting professional refresh

```

Figura 7.12: Ejecución de los procesos automáticos en Render

Los registros muestran la ejecución del proceso encargado de calcular los datos de los campeones. Este proceso actualiza sus estadísticas y tiers, y genera las builds recomendadas. Los mensajes de inicio y finalización permiten comprobar que la tarea termina sin interrumpir el funcionamiento del backend.

También se comprueba directamente la información almacenada en PostgreSQL. La Figura 7.13 muestra una parte de la tabla de jugadores después de utilizar la aplicación y ejecutar las actualizaciones automáticas. Cada jugador se identifica mediante su PUUID y conserva su Riot ID, plataforma y, cuando se ha sincronizado su perfil, el nivel, el icono y la fecha de la última sincronización.

puuid # varchar(255)	game_name # varchar(255)	last_sync_at ⌚ timestampz(6)	platform # varchar(255)	profile_icon_id # int4	summoner_level # int8	tag_line # varchar(255)
seEKbBaMjW6b1wgmouQf	Althèa	(Null)	EUW1	(Null)	(Null)	9598
itHKb0KwR7Rbah7XaTr3f9c	s1lutyfemoby	(Null)	EUW1	(Null)	(Null)	ΛΟΟΠΔ
O-jjGNMo4Szmc2miZdtK	sans emploi	(Null)	EUW1	(Null)	(Null)	EUW
DKQQLR2E58bUpCG9YHqb	KlappaEUW	(Null)	EUW1	(Null)	(Null)	55122
CAfwwIR0eOiolM-_m-LSn	Riveo	(Null)	EUW1	(Null)	(Null)	007
7qr56oMTR5v2i1zziT8hTOYl	xiaolongxia	(Null)	EUW1	(Null)	(Null)	3223
IPrBKKEIIVU8_pblilhuitbAuZ	kiss the bush	(Null)	EUW1	(Null)	(Null)	S2XXX
iZUwziBjDmAOdOvr1-Rq_	Katze	2026-08-08 15:24:01.028372	EUW1	29	101	myao
Yjaqi1-N9q7BwqvZ_xlV6i7l	blueskies777	2026-08-08 15:24:04.059892	EUW1	3798	496	EUW
81so0ftQqFNOUbkjHdR	G2 Labrov	2026-08-08 15:24:04.738812	EUW1	0	528	8085
dbWWByxhTm7377p3wZ8l	T1 Smash	2026-08-08 15:24:04.867247	EUW1	7085	77	2006
vC_7_zhZq77DhgTLDYSjDl	Guilhoto	2026-08-08 15:24:05.350714	EUW1	4025	51	Messi
d4mYxYSFiwjHWez7fNDdf	yebit	2026-08-08 15:24:04.283235	EUW1	7083	1123	haha
QTMWpWamfclMxmWbPl	hovinko z kose	2026-08-08 15:24:05.637538	EUW1	7146	1126	marek

Figura 7.13: Jugadores almacenados en PostgreSQL

Algunos jugadores muestran valores vacíos porque se han almacenado al aparecer como participantes en partidas de otros jugadores, pero su perfil todavía no se ha buscado directamente. Cuando uno de estos jugadores se

consulta por primera vez, el backend completa su información sin crear un duplicado.

Estas comprobaciones confirman que las tareas programadas se ejecutan correctamente y que la información obtenida y calculada por el backend queda almacenada en PostgreSQL para ser reutilizada en consultas posteriores.

7.3. Cumplimiento de requisitos

Después de ejecutar los principales casos de uso, se comparan los resultados obtenidos con los requisitos definidos al inicio del proyecto. Las pruebas anteriores permiten comprobar las funciones relacionadas con jugadores, campeones, profesionales y procesos automáticos. Para completar la evaluación, también se revisan la adaptación de la interfaz y el tratamiento de errores.

La adaptabilidad se comprueba utilizando una pantalla con dimensiones reducidas. Como se observa en la Figura 7.14, los elementos se reorganizan a una sola columna y las funciones siguen siendo accesibles sin necesidad de desplazarse horizontalmente para ver la doble columna original.



Figura 7.14: Adaptación de la interfaz a una pantalla móvil

La gestión de errores se valida realizando una búsqueda de un jugador inexistente. El frontend muestra un mensaje de error legible por el usuario y mantiene el buscador disponible para que pueda corregir los datos e intentarlo de nuevo, como se puede ver en la Figura 7.15.

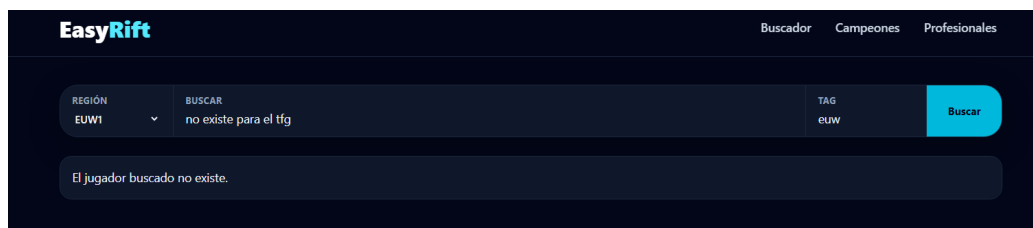


Figura 7.15: Búsqueda incorrecta

La Tabla 7.1 relaciona los requisitos con las pruebas realizadas. Se agrupan aquellos que se comprueban mediante el mismo recorrido para evitar repetir las descripciones explicadas previamente en el capítulo 4.

Requisitos	Evidencia	Resultado
RF1–RF5, RF9, RFA2 y RFA5	Las pruebas de búsqueda y consulta de perfiles comprueban la información básica, los rangos, las estadísticas, el historial, los detalles de partidas, el estado actual y la actualización del jugador.	Cumplido
RF6, RFA1, RFA4, RFA6 y RFA8	El listado y la página de detalle de campeones permiten consultar la clasificación, las estadísticas, las recomendaciones y las builds profesionales según el rol seleccionado.	Cumplido
RF7 y RFA7	Las pruebas permiten consultar builds profesionales desde los campeones y acceder al listado y los perfiles de los profesionales.	Cumplido
RF8	El panel de información permite consultar el nombre y la descripción de los objetos tanto con el cursor como mediante una pantalla táctil.	Cumplido
RFA3	Los registros de Render muestran la ejecución periódica de la actualización de profesionales y del cálculo de los datos de campeones.	Cumplido
RNF1, RNF3 y RNF6	La navegación es clara para diferentes tamaños de pantalla y los errores se muestran mediante mensajes comprensibles.	Cumplido

Continúa en la siguiente página

Requisitos	Evidencia	Resultado
RNF2, RNF4 y RNF5	Los datos se conservan en PostgreSQL y la aplicación mantiene separados el frontend, el backend y la persistencia tanto en local como en producción.	Cumplido
RNF7	La versión final se encuentra desplegada y puede consultarse mediante el dominio https://easyrift.es .	Cumplido

Tabla 7.1: Resumen del cumplimiento de requisitos

Los resultados confirman que la versión final cumple todos los requisitos definidos. Las funciones desarrolladas en el MVP se mantienen y las añadidas en los incrementos posteriores funcionan correctamente en la aplicación desplegada.

7.4. Limitaciones

La principal limitación de EasyRift se encuentra en la API de Riot Games. La clave utilizada permite realizar un máximo de cien solicitudes cada dos minutos. Esta restricción afecta inicialmente a la creación de la plantilla de profesionales. Para registrar cada profesional nuevo se realizan dos peticiones: una para obtener su PUUID mediante el Riot ID y otra para consultar sus datos básicos. Por este motivo, la plantilla se limita a 48 profesionales, ya que su inicialización requiere 96 solicitudes y permanece por debajo del límite.

La carga de partidas se realiza posteriormente mediante el planificador. Cada sincronización solicita los identificadores de las partidas recientes y consulta individualmente aquellas que todavía no están almacenadas. Para controlar este consumo, se cargan como máximo veinte partidas por jugador. Si se alcanza el límite de Riot durante la actualización de los profesionales, el proceso se detiene y continúa en la siguiente ejecución.

Riot permite localizar jugadores de rangos altos y consultar posteriormente sus partidas, lo que podría utilizarse para formar una muestra más amplia. No obstante, recopilar miles de partidas exigiría realizar peticiones continuamente en tandas separadas por el límite de la clave utilizada. Los procesos automáticos alcanzarían constantemente dicho límite y competirían con las consultas realizadas por los usuarios, que podrían no disponer de

llamadas para buscar o actualizar sus perfiles.

En consecuencia, las estadísticas, los tiers y las builds recomendadas se calculan únicamente con las partidas disponibles en PostgreSQL. Estas proceden de los profesionales actualizados automáticamente y de los perfiles buscados por los usuarios. La muestra obtenida permite comprobar que el sistema funciona, pero no es suficiente para que la información sea del todo fiable. Además, al comenzar un nuevo parche, puede no existir todavía una cantidad suficiente de partidas para generar recomendaciones para algunas combinaciones de campeón y rol.

La plantilla profesional se inicializa mediante el archivo `professionals.json`. Para añadir un jugador, es necesario conocer previamente una de sus cuentas y añadir manualmente su Riot ID, equipo, liga y rol. Durante la evaluación existen dos nuevos fichajes procedentes de Asia cuyas cuentas todavía no son conocidas, por lo que no pueden inicializarse en la aplicación. El archivo también debe actualizarse manualmente cuando se producen fichajes, cambios de equipo o modificaciones en las cuentas utilizadas.

Otra limitación se encuentra en la gestión del esquema de PostgreSQL. La propiedad `spring.jpa.hibernate.ddl-auto` utiliza el valor `update`, por lo que Hibernate intenta adaptar automáticamente las tablas al iniciar una nueva versión del backend. Esta configuración es muy cómoda durante el desarrollo, pero puede ser arriesgada en producción, ya que cambios como renombrar columnas, modificar restricciones o eliminar atributos no siempre pueden aplicarse de forma segura.

La evaluación no incluye un estudio de usabilidad con una muestra amplia y diversa de usuarios, ya que no se dispone de un grupo externo dedicado a realizar las pruebas. Se ha proporcionado acceso a la aplicación a varios usuarios reales y se utiliza su opinión para corregir problemas y mejorar algunos elementos de la interfaz. No obstante, este proceso es una validación informal y no permite obtener resultados concluyentes sobre la facilidad de uso.

La aplicación se comprueba mediante las pruebas funcionales descritas anteriormente, pero el backend no cuenta con una batería de pruebas unitarias para verificar sus servicios de forma aislada. Además, el límite de peticiones y la dependencia de la API de Riot dificultan la ejecución repetida de pruebas de integración contra el servicio real.

Nota: Esto te lo mencioné en la reunión que tuvimos en su día. No sé cuánta importancia le dará el tribunal. Ana no las hizo tampoco y el tribunal no dijo nada, pero si tú crees que son necesarios

los tests de JUnit, los puedo hacer sin problema. De momento lo he planteado como un incremento futuro, puedes verlo en el siguiente capítulo.

Nota: La cosa es que las pruebas profesionalmente suelen hacerse haciendo lo que se conoce como CI/CD. Hay varias opciones, pero la más común es con github actions. Eso lo que hace es preparar pruebas que se pasan automáticamente antes de cada despliegue para comprobar que todo lo anterior no se ha roto con lo nuevo. Es meterse en un berenjenal, pero si el tribunal valora que haya tests unitarios en el backend se pueden hacer sin meter github actions.

Nota: En TW nos enseñaron a probar haciendo peticiones HTTP y viendo el resultado JSON de la API a mano. En MDAI nos hicieron hacer tests unitarios de los repositorios (para ver que se hacían las cosas bien en la BD) y de los servicios (para ver que la lógica se implementaba bien). Las pruebas que hice y nuestro son básicamente funcionales en la propia web ya que así se ve fácil si funciona o no.

Nota: Para ver la BD usé una herramienta llamada Navicat que te permite ver cómo cambian los registros de las tablas en tiempo real (y modificarlos). En la captura de las pruebas sale una figura de navicat. Para probar el backend antes de hacer el frontend, miré a mano el json que devolvía la API; creo que te lo llegué a enseñar en aquella reunión. No obstante, esto no lo he puesto porque es un poco chapuza. Quiero saber tu opinión sobre si debo meter los tests o no son tan importantes.

Tampoco se realizan pruebas de carga con una cantidad elevada de usuarios simultáneos. EasyRift continúa pendiente de la aprobación de Riot Games, por lo que se encuentra como una versión de prueba.

A pesar de estas limitaciones, la evaluación confirma que los componentes funcionan correctamente en el entorno desplegado y que se cumplen los requisitos establecidos para la versión final. La aplicación permite completar los recorridos previstos y proporciona una base funcional sobre la que pueden desarrollarse nuevas mejoras.

Nota: En el correo donde me diste el índice me dijiste lo siguiente: "Puedes extenderte más o menos, incluyendo además un

análisis crítico del desarrollo incremental que has seguido, las limitaciones del sistema...”. No sé si está bien así el capítulo o tengo que meter un apartado llamado análisis crítico, he sido crítico en cuanto a limitaciones, pruebas... pero no hay como tal un apartado de análisis.

Capítulo 8

Conclusiones y trabajo futuro

En este capítulo se presentan las conclusiones obtenidas tras el desarrollo de EasyRift y los principales conocimientos adquiridos. También se analizan las aportaciones de la aplicación y se proponen diferentes líneas de trabajo futuro para ampliar sus funciones y prepararla para una posible evolución y su publicación definitiva.

8.1. Conclusiones y conocimientos adquiridos

El desarrollo de EasyRift permite alcanzar el objetivo principal del trabajo: diseñar, implementar y desplegar una aplicación web full-stack para consultar y analizar información de *League of Legends* obtenida mediante la API de Riot Games. La versión final permite buscar jugadores, consultar sus partidas, rangos y estadísticas, analizar campeones, obtener recomendaciones por rol y acceder a información de jugadores profesionales.

Además de las funciones de la propia aplicación, otro de los objetivos del trabajo consiste en completar por primera vez el ciclo de desarrollo de un proyecto, desde su planteamiento inicial hasta su publicación. Durante la carrera, las tecnologías se han trabajado principalmente mediante ejercicios, prácticas o componentes aislados, pero los trabajos realizados no han llegado a desplegarse como una aplicación pública real.

EasyRift permite reunir todos los conocimientos adquiridos a lo largo del grado creando un único sistema formado por frontend, backend y base de datos, abordando también aspectos como el uso de servicios externos, la configuración de entornos, el despliegue, el dominio y el mantenimiento. De esta forma, el proyecto no se limita a implementar una serie de funciones,

sino que realiza el proceso necesario para crear desde cero una aplicación que, tras las ampliaciones y aprobaciones necesarias por parte de Riot, puede evolucionar hasta convertirse en un producto disponible para usuarios reales.

El desarrollo incremental es muy adecuado para construir la aplicación de forma progresiva. El primer incremento proporciona un MVP funcional centrado en las consultas básicas. A partir de esta base, el segundo añade estadísticas y procesos automáticos, mientras que el tercero agrega la clasificación de campeones, las builds recomendadas por rol y los perfiles profesionales. Esta forma de trabajar permite comprobar el funcionamiento de cada versión antes de añadir nuevas funciones y adaptar las decisiones anteriores cuando aparecen nuevos requisitos.

El proyecto también permite entender el funcionamiento de los componentes de una aplicación web. En el backend se adquiere experiencia con Java, Spring Boot, API REST, JPA, Hibernate y tareas programadas. También se trabaja con relaciones entre entidades, consultas agregadas, tratamiento de errores, exclusión mutua y transformación de respuestas externas en información para la aplicación.

La integración con Riot Games permite conocer los problemas que pueden aparecer al depender de una API externa, como los límites de peticiones, los errores en las respuestas y las diferencias entre sus endpoints. También muestra la importancia de reutilizar los datos almacenados para reducir al máximo el número de peticiones.

En el frontend se adquiere experiencia con React, TypeScript, React Router y Tailwind CSS. La interfaz se divide en páginas y componentes reutilizables, obtiene los datos proporcionados por el backend y adapta su distribución a diferentes tamaños de pantalla. El uso de tipos equivalentes a los DTO del backend facilita mantener una comunicación clara entre ambos componentes.

Por último, el despliegue permite trasladar el proyecto del entorno local hasta una aplicación accesible mediante un dominio. Se trabaja con Docker, variables de entorno, PostgreSQL, Render y Vercel, además de configurar la comunicación entre servicios desplegados de forma independiente. Este proceso permite conocer no solo el desarrollo del código, sino también las tareas necesarias para publicar y mantener una aplicación web completa.

EasyRift permite aplicar de forma práctica conocimientos de diseño de bases de datos, desarrollo web, comunicación mediante API, procesamiento de información y despliegue. El resultado es una aplicación funcional que cumple los objetivos establecidos y proporciona una base sobre la que pueden incorporarse nuevas funciones y mejoras técnicas.

8.2. Aportaciones e impacto de EasyRift

EasyRift comparte algunas funciones con plataformas consolidadas de análisis de *League of Legends*, como OP.GG. Ambas permiten consultar jugadores, clasificar campeones y obtener información sobre las builds utilizadas. Sin embargo, EasyRift no pretende competir con una plataforma comercial con millones de usuarios, sino ofrecer una experiencia más sencilla y directa. Además, el proyecto permite desarrollar de forma completa el proceso de obtención, almacenamiento, análisis y presentación de la información.

Una diferencia importante se encuentra en la presentación de las builds recomendadas. Como se observa en la Figura 8.1, OP.GG muestra varias combinaciones iniciales, diferentes grupos de objetos principales y distintas alternativas para completar la build. Este enfoque proporciona más información y permite que un jugador experimentado adapte la elección a la partida, pero también obliga al usuario principiante a interpretar las opciones disponibles y construir por sí mismo la combinación final.

Akali item builds >					
Starter Items			Boots		
		59.43% 75,245 Games	49.45%		79.32% 97,233 Games
		36.67% 46,426 Games	49.8%		15.72% 19,274 Games
Core builds					
	>		>		32.18% 23,770 Games
	>		>		9.9% 7,314 Games
	>		>		7.01% 5,181 Games
	>		>		3.09% 2,284 Games
	>		>		2.8% 2,065 Games
Fourth Item		Fifth Item		Sixth Item	
	53.65% 11,435 Games		53.72% 1,813 Games		51.43% 35 Games
	59.59% 7,748 Games		58.85% 1,633 Games		50% 32 Games
	50.78% 3,588 Games		56.52% 920 Games		42.86% 28 Games
	77.09% 1,790 Games		54.72% 890 Games		69.57% 23 Games
	59.55% 1,382 Games		70.24% 625 Games		53.33% 15 Games

Figura 8.1: Alternativas de objetos mostradas por OP.GG. Fuente: OP.GG [3]

EasyRift está orientada a usuarios que quieren obtener una respuesta sencilla sin dedicar tiempo a estudiar todas las alternativas. Para cada combinación de campeón y rol, selecciona una única build completa de seis objetos, además de los hechizos de invocador y el objeto especial correspondiente. La recomendación se obtiene comparando las configuraciones del parche actual mediante una puntuación que tiene en cuenta su tasa de victorias y su tasa de selección normalizada.

La finalidad no consiste en afirmar que existe una build válida para todas las partidas. La elección óptima puede variar según los rivales, los aliados y el desarrollo de la partida. EasyRift proporciona una opción general y fácil de interpretar, pensada como punto de partida para usuarios que prefieren una recomendación directa frente a un conjunto amplio de posibilidades.

Durante el desarrollo de este trabajo, OP.GG añade una función de recomendación personalizada de campeones en fase beta. Como muestra la Figura 8.2, esta función utiliza los campeones más jugados, los resultados y otras estadísticas del jugador para realizar la recomendación. Su existencia muestra que la recomendación personalizada todavía es una función en evolución dentro de las plataformas de análisis y que existe interés por simplificar la toma de decisiones del usuario.

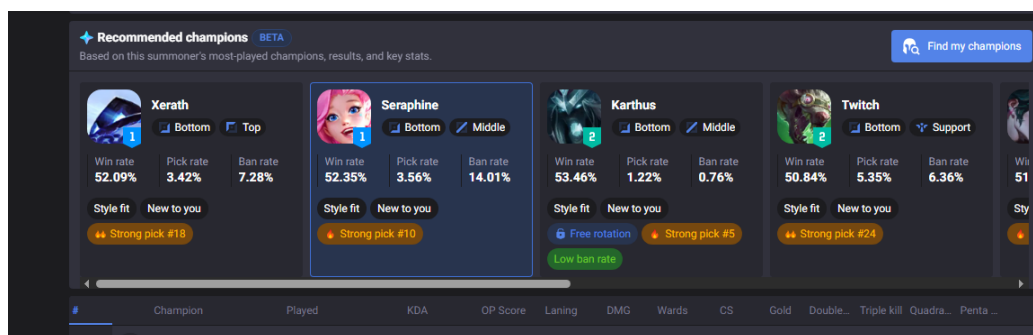


Figura 8.2: Recomendación personalizada de campeones en fase beta de OP.GG. Fuente: OP.GG [4]

EasyRift realiza esta recomendación con un enfoque diferente. En lugar de proponer campeones que el jugador todavía no utiliza, identifica su mejor campeón a partir de sus partidas, combinando los resultados obtenidos y la frecuencia de uso. De esta forma, recomienda un campeón con el que el jugador ya ha obtenido buenos resultados, en lugar de uno que nunca ha utilizado y con el que probablemente tendría menos posibilidades de ganar.

Otra aportación de EasyRift es la información de los jugadores profesionales. Existen plataformas especializadas como LolPros que tienen una base de datos de jugadores profesionales y relacionan a cada jugador con sus cuen-

tas conocidas. Como se puede ver en la Figura 8.3, esta plataforma permite consultar sus cuentas, posiciones, rangos e historial competitivo.

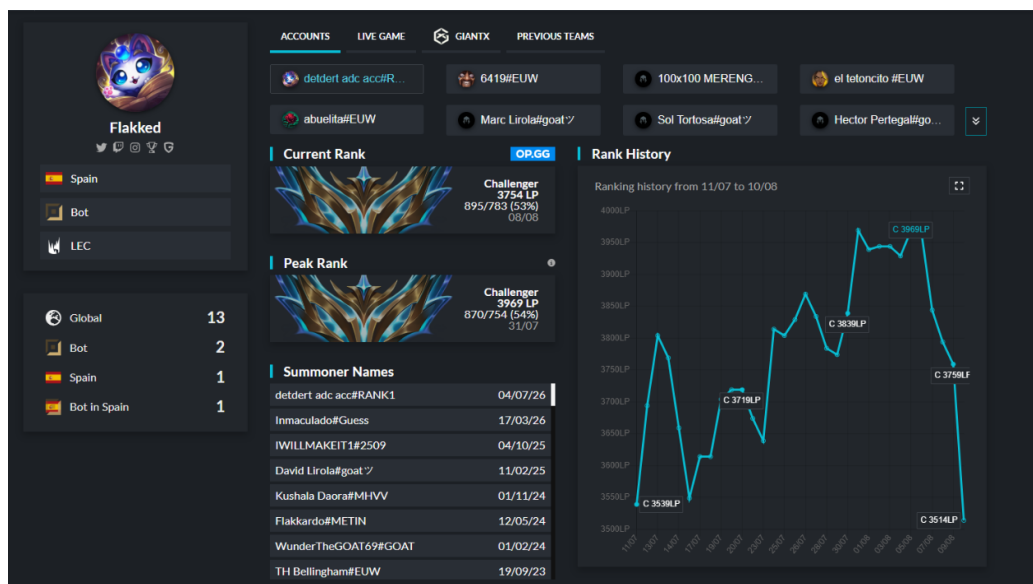


Figura 8.3: Información de un jugador profesional mostrada por LolPros. Fuente: LolPros [5]

EasyRift utiliza otro enfoque y conecta la información del profesional con el análisis de campeones. Los profesionales aparecen agrupados por equipos y cuentan con un perfil propio con sus builds recientes. Además, las builds profesionales se muestran en la página del campeón y se filtran mediante el mismo selector de rol utilizado para la recomendación. El usuario puede pasar directamente desde una build al perfil del profesional que la utiliza y, desde allí, consultar también el perfil normal de su cuenta.

Las principales aportaciones de EasyRift son la generación de una única build completa por campeón y rol, la utilización conjunta de la tasa de victorias y la tasa de selección normalizada, la conexión entre recomendaciones y builds profesionales, y la integración de perfiles de jugadores y profesionales dentro de la misma aplicación.

El principal impacto de EasyRift consiste en facilitar el acceso a esta información a usuarios que disponen de poco tiempo o no tienen experiencia en el videojuego. Con una muestra de partidas mayor y una clave de producción de Riot, el sistema podría generar resultados más representativos y convertirse en una herramienta pública de consulta muy fiable.

El proyecto demuestra que es posible desarrollar desde cero una plataforma de este tipo integrando una API externa como fuente de datos, una base

de datos propia, procesos de análisis para convertir los datos en información y una interfaz web accesible.

8.3. Líneas de trabajo futuro

EasyRift es actualmente un prototipo funcional desplegado y accesible mediante un dominio propio. Esto permite iniciar el proceso de solicitud de una clave de producción, ya que Riot Games pide un prototipo operativo antes de solicitar la aprobación de un producto. La existencia de esta versión no garantiza su aprobación, pero es la base necesaria para presentar la aplicación y demostrar su funcionamiento [12].

Nota: Todos los productos que tengan que ver con Riot Games necesitan su aprobación (es que hasta cuando un equipo ficha a un jugador piden aprobación y tienen que poner "pendiente de aprobación de riot" al anunciarlo). Para darte la clave sin límites (tiene límites pero como si no los tuviera), te piden un prototipo de la aplicación que tiene que estar público (porque te piden el link), pero a su vez, te dicen que no puedes publicarlo hasta que te lo acepten. Esto es una aplicación completa si no tenemos en cuenta los límites tan bajos que tiene, no obstante, para lavarme las manos, he puesto una ventana modal al entrar a la web que dice que es una aplicación en pruebas.

A partir del prototipo desarrollado y tras obtener la clave de producción, la evolución de EasyRift se puede plantear mediante dos nuevos incrementos. El primero ampliaría los datos y las funciones de la aplicación, aprovechando el mayor límite de la nueva clave, mientras que el segundo se centraría en añadir las pruebas y mejoras técnicas necesarias para garantizar su estabilidad antes de la publicación definitiva.

8.3.1. Cuarto incremento: ampliación de datos y funcionalidades

El cuarto incremento se centraría en aprovechar el mayor límite de peticiones para ampliar la cantidad de datos disponibles. Esto permitiría añadir más profesionales y actualizar sus partidas con mayor frecuencia, dejando suficientes llamadas disponibles para las búsquedas y actualizaciones realizadas por los usuarios.

Este aumento del límite también permitiría crear una muestra de datos más amplia. Para ello, se localizarían jugadores de rangos altos y se recopilarían sus partidas clasificatorias en otra tarea periódica. De esta forma, las estadísticas, los tiers y las builds recomendadas no dependerían únicamente de los profesionales y de los perfiles buscados, sino de una cantidad mayor de partidas del parche actual.

La siguiente ampliación sería la paginación del historial. Actualmente, cada sincronización carga un máximo de veinte partidas. Con esta mejora, el usuario podría solicitar partidas adicionales al llegar al final de la lista y consultar partidas más antiguas sin tener que cargar todo el historial de una sola vez.

La clasificación de campeones se podría calcular por separado para cada rol, ya que un mismo campeón puede obtener resultados diferentes según la posición en la que se utilice. También se podrían añadir métricas adicionales como los súbditos eliminados, el oro obtenido, el daño realizado o la puntuación de visión. Además, la aplicación podría recomendar las runas, los hechizos de invocador, el orden en el que mejorar las habilidades y diferentes objetos según el desarrollo de la partida.

Asimismo, se podría ampliar el cálculo del mejor campeón del jugador. En lugar de seleccionar un único campeón general, se podría determinar el mejor para cada rol a partir de las partidas de su historial. Al disponer de más datos mediante la paginación, estas recomendaciones serían más representativas y permitirían al usuario consultar su mejor opción en cada posición.

8.3.2. Quinto incremento: preparación para producción

Durante los primeros incrementos, la prioridad es desarrollar y ampliar el prototipo funcional. Como las entidades, los DTO y los servicios cambian con frecuencia, cada versión se comprueba principalmente mediante pruebas funcionales sobre la aplicación completa. Dentro del alcance del trabajo, las pruebas automatizadas del backend se aplazan hasta tener una versión completa y estable.

En este quinto incremento se añadirían pruebas unitarias con respuestas simuladas y pruebas de integración entre los distintos componentes. También se realizarían pruebas de carga y un estudio de usabilidad. Las pruebas automatizadas se integrarían en un proceso de integración continua y despliegue continuo (CI/CD). Este proceso comprobaría automáticamente cada nueva

versión antes de publicarla y detendría el despliegue si alguna prueba fallase.

También se mejoraría la gestión de la estructura de PostgreSQL. En lugar de depender de la actualización automática realizada por Hibernate, se utilizaría una herramienta especializada para aplicar de forma controlada los cambios de cada versión. Esto reduciría el riesgo de modificar incorrectamente tablas que ya contienen datos.

Para acelerar las consultas más frecuentes, se podría añadir Redis como caché en memoria. Los listados de campeones y profesionales, junto con algunos resultados calculados, se podrían conservar temporalmente en ella. De esta forma, se reducirían las consultas repetidas a PostgreSQL y mejorarían los tiempos de respuesta.

Finalmente, si el número de usuarios aumentase, Render permite ejecutar varias instancias del backend y distribuir las peticiones entre ellas. En ese caso, habría que coordinar los procesos automáticos para evitar que cada instancia ejecutase las mismas tareas programadas. Estas líneas de trabajo permitirían transformar el prototipo actual en una aplicación más completa, estable y preparada para aumentar progresivamente su número de usuarios.

8.3.3. Evolución a largo plazo

A más largo plazo, EasyRift podría añadir recomendaciones para otros modos de juego de LoL, incluidos los modos temporales que Riot Games añade periódicamente. Para ello, sería necesario adaptar el modelo de datos, ya que estos modos pueden utilizar reglas, mapas, objetos o composiciones de equipo diferentes. La aplicación podría separar sus estadísticas y recomendaciones para evitar mezclar partidas que pertenecen a modos con características distintas.

El planteamiento de EasyRift también podría servir como base para desarrollar aplicaciones orientadas a otros MOBA o videojuegos competitivos que cuenten con una API con información sobre jugadores y partidas. Parte de la arquitectura podría mantenerse, como la separación entre frontend, backend y base de datos o el procesamiento periódico de información. Sin embargo, sería necesario adaptar el modelo de datos, los cálculos y la interfaz a cada videojuego. Por este motivo, se plantea como una evolución independiente de los incrementos anteriores.

Nota: Cuando Ana vio que estaba haciendo el TFG de un videojuego, me copió la idea y cambió su tema. Cuando lo presentó en julio, le preguntaron específicamente si su idea podía escalar a

otros juegos. Me adelanto a la pregunta con este apartado, aunque sea pequeño.

Bibliografía

- [1] Stack Overflow, “Technology: 2025 stack overflow developer survey.” <https://survey.stackoverflow.co/2025/technology>, 2025. Consultado: 11-08-2026.
- [2] Riot Games, “Riot games api reference.” <https://developer.riotgames.com/apis>, 2026. Consultado: 11-08-2026.
- [3] OP.GG, “Akali build: Items, runes and counters.” <https://op.gg/es/lol/champions/akali/build>, 2026. Consultado: 11-08-2026.
- [4] OP.GG, “Recommended champions.” <https://op.gg/es/lol/summoners/euw/iFran28-EUW/champions>, 2026. Consultado: 11-08-2026.
- [5] LolPros, “Flakked: Professional league of legends player profile.” <https://lolpros.gg/player/flakked>, 2026. Consultado: 11-08-2026.
- [6] Amazon Web Services, “¿cuál es la diferencia entre el front end y back end en el desarrollo de aplicaciones?.” <https://aws.amazon.com/es/compare/the-difference-between-frontend-and-backend/>, 2026. Consultado: 11-08-2026.
- [7] J. M. Beas, “Desarrollo iterativo e incremental.” <https://blog.jmbeas.es/2019/09/22/desarrollo-iterativo-e-incremental/>, 2019. Consultado: 11-08-2026.
- [8] Universidad Internacional de La Rioja, “Producto mínimo viable (mvp): ¿qué es?.” <https://www.unir.net/revista/empresa/mvp-producto-minimo-viable/>, 2024. Consultado: 11-08-2026.
- [9] OP.GG, “Acerca de op.gg: nuestra historia y crecimiento.” <https://op.gg/es/lol/about>, 2026. Consultado: 11-08-2026.

- [10] Blitz, “Blitz.” <https://www.linkedin.com/company/theblitzapp/about/>, 2026. Consultado: 11-08-2026.
- [11] LolPros.GG, “Frequently asked questions.” <https://lolpros.gg/about/faq>, 2026. Consultado: 12-08-2026.
- [12] Riot Games, “League of legends developer documentation.” <https://developer.riotgames.com/docs/lol>, 2026. Consultado: 11-08-2026.
- [13] Red Hat, “¿qué es una api rest?.” <https://www.redhat.com/es/topics/api/what-is-a-rest-api>, 2023. Consultado: 11-08-2026.
- [14] Universidad Internacional de La Rioja, “¿qué son las single page application (spa) y cómo funcionan?.” <https://www.unir.net/revista/ingenieria/single-page-application-que-es/>, 2026. Consultado: 11-08-2026.
- [15] Vite, “Getting started.” <https://vite.dev/guide/>, 2026. Consultado: 11-08-2026.
- [16] Docker, “What is docker?.” <https://docs.docker.com/get-started/docker-overview/>, 2026. Consultado: 11-08-2026.
- [17] Riot Games Competitive Operations, “League of legends.” <https://competitiveops.riotgames.com/es-ES/league-of-legends>, 2026. Consultado: 12-08-2026.
- [18] M. Goodwin, “¿qué es una api (interfaz de programación de aplicaciones)?.” <https://www.ibm.com/es-es/think/topics/api>, 2026. Consultado: 11-08-2026.
- [19] IBM, “¿qué es un endpoint de api?.” <https://www.ibm.com/es-es/think/topics/api-endpoint>. Consultado: 12-08-2026.
- [20] R. Pyro and R. cjhamster, “Transitioning from summoner names to riot ids.” <https://www.riotgames.com/en/news/summoner-name-riot-ID>, 2023. Consultado: 11-08-2026.
- [21] R. Phlox, R. Cashmiir, and R. Riru, “Lol pls: Preseason 2023 jungle changes.” <https://www.leagueoflegends.com/en-us/news/dev/lol-pls-preseason-2023-jungle-changes/>, 2022. Consultado: 11-08-2026.

- [22] Dashijador and R. C. me KT, “/dev: Wasd controls are on the way.” <https://www.leagueoflegends.com/en-us/news/dev/dev-wasd-controls-are-on-the-way/>, 2025. Consultado: 11-08-2026.
- [23] Render, “Render documentation.” <https://render.com/docs>, 2026. Consultado: 08-08-2026.
- [24] Render, “Scaling render services.” <https://render.com/docs/scaling>, 2026. Consultado: 08-08-2026.
- [25] Amazon Web Services, “¿qué es el uso compartido de recursos entre orígenes (cors)?.” <https://aws.amazon.com/es/what-is/cross-origin-resource-sharing/>. Consultado: 12-08-2026.
- [26] Vercel, “Vercel documentation.” <https://vercel.com/docs>, 2026. Consultado: 08-08-2026.
- [27] Vicerrectorado de Transformación Digital, Universidad de Extremadura, “Guía de uso responsable de inteligencia artificial generativa para estudiantado.” <https://vrdigital.unex.es/wp-content/uploads/sites/28/2025/11/IA-UEx-Estudiantado-v2.pdf>, 2025. Consultado: 11-08-2026.

Anexo A

Uso responsable de herramientas de inteligencia artificial

Durante el desarrollo de este trabajo, se han utilizado herramientas de inteligencia artificial generativa como apoyo en distintas tareas. Una de ellas ha sido la búsqueda de referencias útiles, ya que ChatGPT no solo permite buscar información, sino también localizar fuentes a las que acudir para consultarla.

Durante la implementación también se ha utilizado la predicción de código de GitHub Copilot. Esta herramienta propone continuaciones a medida que se escribe, utilizando como contexto el código cercano. Su uso permite escribir con mayor rapidez estructuras habituales, como bucles y condiciones, además de documentación y otras tareas repetitivas. Aunque las sugerencias no siempre son correctas y deben revisarse antes de aceptarlas, son útiles para agilizar este tipo de trabajos.

La parte relacionada con el frontend no se ha estudiado durante el grado y ha supuesto un reto personal dentro del proyecto. Para aprender React, TypeScript y Tailwind CSS se ha seguido un curso en línea y se ha utilizado Codex como apoyo. Esta herramienta puede acceder a los archivos locales del proyecto, por lo que dispone de todo el contexto de la arquitectura. Esto ha sido útil para aprender conceptos nuevos, mantener una organización coherente y recibir explicaciones adaptadas a las dudas surgidas durante el desarrollo.

La memoria se ha redactado en Overleaf, que incluye una herramienta de corrección automática mediante inteligencia artificial. Esta función ayu-

da a detectar problemas de puntuación y propone cambios para reemplazar expresiones informales o mejorar la claridad de la redacción, de una forma similar a los correctores ortográficos habituales de Word.

En ningún caso la inteligencia artificial sustituye el trabajo del ingeniero, ya que las decisiones de diseño, la implementación y la comprobación de los resultados siguen siendo responsabilidad del autor. Las herramientas se utilizan como apoyo para aprender, buscar información, acelerar tareas repetitivas y revisar el trabajo realizado. Utilizadas de forma crítica y responsable, pueden ser de gran utilidad para adquirir conocimientos y mejorar el proceso de desarrollo [27].

Nota: Queda raro decir "del autor", pero no puedo ponerlo en primera persona, así que no sé cómo.

Nota: Fui a ver las presentaciones de los TFGs de informática de julio, y Enrique le preguntó directamente a Ana cómo había usado la IA en el trabajo, ya que en la memoria no lo aclaraba. No sé si esto queda demasiado forzado/descarado, ya que algunos profesores son anti IA (Luis, el tribunal en concreto son todos pro IA), pero creo que no está mal aclararlo, aunque se sobreentienda.

Nota: Hay un pdf de la UEx sobre la IA, pero creo que en nuestro centro no hay ninguno en concreto. Se supone que, mientras se use como apoyo, no hay problema, pero creo que depende mucho de cada profesor. Te lo dejo aquí (Página 12). <https://vrdigital.unex.es/wp-content/uploads/sites/28/2025/11/IA-UEx-Estudiantado-v2.pdf>